

Linked Lists & Strings

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

August 21, 2026

Contents

1	Linked Lists	1
1.1	Pointers, Two Pointers, and the Dummy Node	1
1.2	Singly Linked List: Construction and Basic Operations	2
1.3	Doubly Linked List: Construction and Operations	3
1.4	Reverse a Doubly Linked List	5
1.5	Middle of a Linked List	6
1.6	Reverse a Linked List (Iterative)	8
1.7	Reverse a Linked List (Recursive)	9
1.8	Detect a Loop in a Linked List	11
1.9	Starting Point of a Loop	13
1.10	Length of a Loop	14
1.11	Check if a Linked List is a Palindrome	16
1.12	Segregate Odd and Even Position Nodes	18
1.13	Remove the Nth Node from the End	19
1.14	Delete the Middle Node	21
1.15	Sort a Linked List	22
1.16	Sort a Linked List of 0s, 1s, and 2s	24
1.17	Intersection Point of Two Linked Lists	26
1.18	Add One to a Number Represented by a Linked List	27
1.19	Add Two Numbers Represented by Linked Lists	29
1.20	Delete All Occurrences of a Key in a Doubly Linked List	31
1.21	Find Pairs with a Given Sum in a Sorted DLL	32
1.22	Remove Duplicates from a Sorted DLL	34
1.23	Reverse Nodes in Groups of K	36
1.24	Rotate a Linked List	37
1.25	Flatten a Linked List	39
1.26	Clone a Linked List with Random Pointers	41
2	Strings	43
2.1	From Character Counting to Border Theory	43
2.2	Remove Outermost Parentheses	44
2.3	Reverse Words in a String	45
2.4	Largest Odd Number in a String	46
2.5	Longest Common Prefix	47
2.6	Isomorphic Strings	49
2.7	Check if One String is a Rotation of Another	50
2.8	Check if Two Strings are Anagrams	51
2.9	Sort Characters by Frequency	53
2.10	Maximum Nesting Depth of Parentheses	54
2.11	Roman Numeral to Integer	55
2.12	Implement Atoi	57
2.13	Longest Palindromic Substring (Expand Around Centre)	58
2.14	Sum of Beauty of All Substrings	59
2.15	Count and Say	61
2.16	Minimum Bracket Reversals to Balance	62
2.17	Rabin–Karp Pattern Matching	64

2.18 Z-Function and Pattern Matching	65
2.19 KMP Algorithm and the LPS Array	67
2.20 Shortest Palindrome	69
2.21 Longest Happy Prefix	70

Chapter 1

Linked Lists

*A linked list trades the array's random access for something arrays cannot do cheaply: rearranging elements by rewiring pointers instead of shifting memory. Almost every linked-list problem is won or lost on two techniques – the two-pointer (slow/fast) trick and the disciplined manipulation of **next** pointers – plus the habit of using a dummy head node so the “first element” is never a special case.*

1.1 Pointers, Two Pointers, and the Dummy Node

A node holds a value and a pointer to the next node; the list is a chain ending in `nullptr`. There is no index, so everything is done by walking pointers – which is why a handful of pointer patterns solve the whole chapter.

Three Habits That Solve Most List Problems

- **Slow/fast (tortoise–hare):** advance one pointer by one and another by two. This finds the middle, detects a cycle, and locates the n -th node from the end in a single pass.
- **Rewire, don't copy:** to reverse, reorder, or remove nodes, change **next** pointers; keep a **prev** and a **next** handle so you never lose the rest of the list.
- **Dummy head:** allocate a throwaway node before the real head so insertion/deletion at the front is not a special case – return `dummy.next` at the end.

How to Recognise the Technique

Linked-List Keywords

- “Middle,” “cycle,” “ n -th from end,” “loop length” – slow/fast two pointers.
- “Reverse,” “reorder,” “swap in groups,” “rotate” – pointer rewiring, often with a dummy node.
- “Palindrome,” “add two numbers,” “intersection” – reverse or align sub-lists, then walk in step.
- “Sort,” “0s/1s/2s” – merge sort on the list, or pointer partitioning.
- “Flatten,” “clone with random pointer” – recursion or an interleaving trick that reuses the list's own structure.
- **Doubly linked list** – an extra **prev** pointer makes backward traversal and $O(1)$ deletion possible.

Throughout, a singly linked node is `struct ListNode { int val; ListNode* next; }` and a doubly linked node additionally carries `ListNode* prev`. Every listing is a complete, compilable program that builds a small list, runs the operation, and prints the result.

1.2 Singly Linked List: Construction and Basic Operations

Problem Statement. Implement the foundational operations on a singly linked list: build from an array, traverse, find the length, search for a value, insert at the head or tail, and delete a value.

Pattern Recognition

No pattern recognition is needed here – this is the machinery every later section rests on. The one habit to internalise is keeping a `prev` handle while walking, so a node can be unlinked or a new node spliced in without losing the rest of the chain.

Walking with a pointer

Each node stores a value and a pointer to the next node; the last node points to `nullptr`. Traversal is a loop that follows `next` until null. Insertion at the head is $O(1)$ (point the new node at the old head). Insertion at the tail and deletion require walking to the right spot, keeping the previous node so its `next` can be rewired.

Keep the Previous Node When You Might Rewire

Deletion and mid-list insertion both need to change the `next` of the node *before* the target. Tracking `prev` as you traverse makes these $O(1)$ once the position is found, with no re-walk.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val;
6      ListNode* next;
7      ListNode(int v) : val(v), next(nullptr) {}
8  };
9
10 ListNode* build(const vector<int>& a) {           // build from array
11     ListNode dummy(0), *tail = &dummy;
12     for (int x : a) { tail->next = new ListNode(x); tail = tail->next;
13     }
14     return dummy.next;
15 }
16
17 int length(ListNode* head) {
18     int n = 0;
19     for (ListNode* c = head; c; c = c->next) n++;
20     return n;
21 }
22
23 bool search(ListNode* head, int key) {
24     for (ListNode* c = head; c; c = c->next) if (c->val == key) return
25         true;
26     return false;
27 }
28
29 ListNode* insertHead(ListNode* head, int v) {
30     ListNode* node = new ListNode(v);

```

```

29     node->next = head;
30     return node;                                // new head
31 }
32
33 ListNode* deleteValue(ListNode* head, int key) {
34     ListNode dummy(0); dummy.next = head;
35     ListNode* prev = &dummy;
36     while (prev->next && prev->next->val != key) prev = prev->next;
37     if (prev->next) prev->next = prev->next->next;    // unlink
38     return dummy.next;
39 }
40
41 void print(ListNode* head) {
42     for (ListNode* c = head; c; c = c->next) cout << c->val << " ";
43     cout << "\n";
44 }
45
46 int main() {
47     ListNode* head = build({1, 2, 3, 4});
48     print(head);                                // 1 2 3 4
49     cout << "length " << length(head) << ", search 3? " << search(head,
50         3) << "\n";
51     head = insertHead(head, 0);
52     head = deleteValue(head, 3);
53     print(head);                                // 0 1 2 4
54 }

```

Complexity Analysis

Time Complexity. $O(1)$ head insert; $O(n)$ for length, search, tail insert, and delete.
Space Complexity. $O(1)$ extra (plus the list itself).

Key Takeaway

The dummy node plus a **prev** handle removes the “deleting the head” special case and is the scaffolding for nearly every operation in this chapter. Master this template before the pointer-heavy problems ahead.

1.3 Doubly Linked List: Construction and Operations

Problem Statement. Implement a doubly linked list (DLL): build it, insert before or after a node, and delete a node – all in $O(1)$ once the node is in hand, thanks to the backward **prev** pointer.

Pattern Recognition

A DLL adds a **prev** pointer to each node. The pay-off is $O(1)$ deletion of a known node (no need to walk to find its predecessor) and easy backward traversal – which is exactly why LRU caches and browser-history structures use it.

Two pointers per node

Each node stores **prev** and **next**. Inserting a node x between a and b rewires four pointers: $a.next = x$, $x.prev = a$, $x.next = b$, $b.prev = x$. Deleting a node x splices its neighbours

together: $x.\text{prev.next} = x.\text{next}$ and $x.\text{next.prev} = x.\text{prev}$, with null checks at the ends. Because a node knows its own predecessor, deletion needs only the node itself.

The prev Pointer Buys $O(1)$ Deletion

In a singly linked list, deleting a node requires its predecessor, found by an $O(n)$ walk. The prev pointer stores that predecessor, so any node can be removed in constant time – the feature that makes DLLs the backbone of caches.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Node {
5      int val;
6      Node* prev;
7      Node* next;
8      Node(int v) : val(v), prev(nullptr), next(nullptr) {}
9  };
10
11 Node* build(const vector<int>& a) {
12     Node dummy(0), *tail = &dummy;
13     for (int x : a) {
14         Node* node = new Node(x);
15         tail->next = node; node->prev = tail;
16         tail = node;
17     }
18     Node* head = dummy.next;
19     if (head) head->prev = nullptr;
20     return head;
21 }
22
23 Node* deleteNode(Node* head, Node* x) {           // O(1) unlink
24     if (x->prev) x->prev->next = x->next;
25     else head = x->next;                           // x was the head
26     if (x->next) x->next->prev = x->prev;
27     return head;
28 }
29
30 void insertAfter(Node* a, int v) {                 // splice x between a
31     // and a->next
32     Node* x = new Node(v);
33     x->next = a->next; x->prev = a;
34     if (a->next) a->next->prev = x;
35     a->next = x;
36 }
37
38 void print(Node* head) {
39     for (Node* c = head; c; c = c->next) cout << c->val << " ";
40     cout << "\n";
41 }
42
43 int main() {
44     Node* head = build({1, 2, 3});
45     insertAfter(head, 9);                           // 1 9 2 3
46     print(head);

```

```

46     head = deleteNode(head, head->next);           // remove the 9
47     print(head);                                   // 1 2 3
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(1)$ per insert/delete of a known node; $O(n)$ to build. **Space Complexity.** $O(1)$ extra per operation.

Key Takeaway

A doubly linked list is a singly linked list with a backward pointer, trading a little memory for $O(1)$ deletion and two-way traversal. Keep the four-pointer insert and two-pointer delete rewiring precise – off-by-one null handling at the ends is the usual bug.

1.4 Reverse a Doubly Linked List

Problem Statement. Reverse a doubly linked list in place so its head becomes the tail and vice versa, returning the new head.

Pattern Recognition

Because each DLL node already knows both neighbours, reversing is just **swapping every node's prev and next** pointers. After swapping all of them, the old tail – whose **next** is now null – is the new head.

Swap the two pointers at every node

Walk the list; at each node exchange its **prev** and **next**. Once swapped, to advance you must follow the *old next*, which is now stored in **prev** – so move via the just-swapped **prev**. When you fall off the end, the last node you swapped is the new head. No new nodes are created; only pointers are flipped.

Reversal Is Local Pointer Swapping

In a DLL, the global reversal decomposes into an identical local operation at each node: swap its two links. Because the links are symmetric, doing this everywhere reverses the whole list without tracking a running **prev** as a singly linked reversal must.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Node {
5      int val;
6      Node* prev;
7      Node* next;
8      Node(int v) : val(v), prev(nullptr), next(nullptr) {}
9  };
10
11 Node* build(const vector<int>& a) {
12     Node dummy(0), *tail = &dummy;

```

```

13     for (int x : a) { Node* n = new Node(x); tail->next = n; n->prev =
        tail; tail = n; }
14     Node* head = dummy.next; if (head) head->prev = nullptr;
15     return head;
16 }
17
18 Node* reverse(Node* head) {
19     Node* curr = head;
20     Node* last = head;
21     while (curr) {
22         last = curr;
23         swap(curr->prev, curr->next);           // flip both links
24         curr = curr->prev;                       // old next is now in
                                                prev
25     }
26     return last;                               // old tail = new
        head
27 }
28
29 void print(Node* head) {
30     for (Node* c = head; c; c = c->next) cout << c->val << " ";
31     cout << "\n";
32 }
33
34 int main() {
35     Node* head = build({1, 2, 3, 4});
36     head = reverse(head);
37     print(head);                               // 4 3 2 1
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$ – only pointer swaps.

Key Takeaway

Reversing a DLL is swapping **prev** and **next** at every node – the symmetric structure makes global reversal a local operation. Contrast with the singly linked reversal (next section) which must thread a running **prev** because there is no backward pointer.

1.5 Middle of a Linked List

Problem Statement. Given the head of a singly linked list, return its middle node in a single pass. For even length, return the second of the two middle nodes.

Pattern Recognition

“Find the middle in one pass” is the **slow/fast two-pointer** (tortoise-hare) technique: move one pointer one step and another two steps; when the fast pointer reaches the end, the slow pointer is at the middle.

Two speeds, one pass

Start both pointers at the head. Each iteration advances **slow** by one node and **fast** by two. When **fast** (or **fast->next**) runs off the end, **slow** has travelled exactly half as far – it sits

at the middle. This avoids first computing the length and then re-walking, doing everything in one traversal.

Half Speed Lands on the Middle

Because `fast` covers twice the distance of `slow`, by the time `fast` finishes the list `slow` has covered half of it. The loop condition (`fast && fast->next`) chooses the second middle for even lengths.

Algorithm

1. `slow = fast = head`.
2. While `fast` and `fast->next` are non-null: advance `slow` by one, `fast` by two.
3. Return `slow`.

Dry run

Dry Run. List $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

- `slow=1, fast=1`. Step: `slow=2, fast=3`. Step: `slow=3, fast=5`.
- `fast->next` is null, stop. Middle = 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* middleNode(ListNode* head) {
16     ListNode* slow = head;
17     ListNode* fast = head;
18     while (fast && fast->next) {
19         slow = slow->next;
20         fast = fast->next->next;
21     }
22     return slow;
23 }
24
25 int main() {
26     ListNode* head = build({1, 2, 3, 4, 5});
27     cout << middleNode(head)->val << "\n";           // 3
28     ListNode* even = build({1, 2, 3, 4});
29     cout << middleNode(even)->val << "\n";           // 3 (second middle)
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$.

Key Takeaway

The slow/fast two-pointer finds the middle in one pass and is the single most reused linked-list trick: cycle detection, n -th-from-end, and palindrome checks all start from it. Learn its even/odd termination precisely.

1.6 Reverse a Linked List (Iterative)

Problem Statement. Reverse a singly linked list in place and return the new head, using $O(1)$ extra space.

Pattern Recognition

Reversing a singly linked list is the archetypal **pointer-rewiring** loop: walk the list carrying three pointers – **prev**, **curr**, **next** – and at each node flip **curr->next** to point backward at **prev**.

Flip each link, remember the rest

The danger in rewiring is losing the remainder of the list. So before flipping **curr->next** to **prev**, save **curr->next** in a temporary **next**. Then set **curr->next = prev**, advance **prev = curr**, and **curr** to the saved **next**. When **curr** becomes null, **prev** is the new head (the old tail).

Save Next Before You Overwrite It

Each step overwrites **curr->next**, which is the only link to the rest of the list – so it must be stashed first. This save-flip-advance trio is the canonical in-place reversal and recurs inside reverse-in-groups and palindrome checks.

Algorithm

1. **prev** = null, **curr** = head.
2. While **curr**: save **next** = **curr->next**; set **curr->next** = **prev**; advance **prev** = **curr**, **curr** = **next**.
3. Return **prev**.

Dry run

Dry Run. List $1 \rightarrow 2 \rightarrow 3$.

- **prev**=null, **curr**=1: flip $1 \rightarrow \text{null}$; **prev**=1, **curr**=2.
- flip $2 \rightarrow 1$; **prev**=2, **curr**=3. flip $3 \rightarrow 2$; **prev**=3, **curr**=null.
- New head $3 \rightarrow 2 \rightarrow 1$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* reverse(ListNode* head) {
16     ListNode* prev = nullptr;
17     ListNode* curr = head;
18     while (curr) {
19         ListNode* next = curr->next;    // save the rest
20         curr->next = prev;              // flip
21         prev = curr;                   // advance
22         curr = next;
23     }
24     return prev;                       // new head
25 }
26
27 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
28     cout << "\n"; }
29
30 int main() {
31     ListNode* head = build({1, 2, 3, 4});
32     head = reverse(head);
33     print(head);    // 4 3 2 1
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

Iterative reversal is the save–flip–advance loop with three pointers, $O(1)$ space. It is the most important pointer-manipulation pattern in the chapter – committed to muscle memory, it makes reverse-in- k -groups, palindrome, and add-two-numbers straightforward.

1.7 Reverse a Linked List (Recursive)

Problem Statement. Reverse a singly linked list using recursion, returning the new head.

Pattern Recognition

The recursive reversal expresses the same rewiring differently: reverse the rest of the list first, then fix the link between the current node and its successor on the way back up. The

new head bubbles up unchanged from the deepest call.

Reverse the tail, then flip one link

Recurse to the end: the base case (empty or single node) returns itself as the new head. On the way back, the current node's successor (`head->next`) should now point back at the current node, so set `head->next->next = head` and `head->next = null`. The deepest call's return value – the original last node – is the new head and is passed up unchanged through every frame.

Fix the Link on the Return Trip

Going down, we only recurse; coming up, each frame flips exactly one link between `head` and `head->next`. Because the new head is determined at the very bottom, it is threaded back up untouched while the per-frame flips reverse the pointers.

Algorithm

1. If `head` is null or `head->next` is null, return `head`.
2. `newHead = reverse(head->next)`.
3. `head->next->next = head`; `head->next = null`.
4. Return `newHead`.

Dry run

Dry Run. List $1 \rightarrow 2 \rightarrow 3$.

- Recurse to 3 (base) $\rightarrow$ `newHead = 3`.
- At 2: $3 \rightarrow 2$, $2 \rightarrow \text{null}$. At 1: $2 \rightarrow 1$, $1 \rightarrow \text{null}$.
- Result $3 \rightarrow 2 \rightarrow 1$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* reverse(ListNode* head) {
16     if (!head || !head->next) return head;           // base case
17     ListNode* newHead = reverse(head->next);
18     head->next->next = head;                           // flip one link

```

```

19     head->next = nullptr;
20     return newHead;           // threaded up
    unchanged
21 }
22
23 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
    cout << "\n"; }
24
25 int main() {
26     ListNode* head = build({1, 2, 3, 4});
27     head = reverse(head);
28     print(head);             // 4 3 2 1
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ for the recursion stack (versus $O(1)$ for the iterative form).

Key Takeaway

Recursive reversal reverses the tail then flips one link per return frame, carrying the new head up from the base case. It is more elegant but uses $O(n)$ stack; prefer the iterative version when depth could overflow.

1.8 Detect a Loop in a Linked List

Problem Statement. Given the head of a singly linked list, determine whether it contains a cycle (some node's `next` points back to an earlier node), using $O(1)$ extra space.

Pattern Recognition

“Is there a cycle?” is **Floyd’s tortoise–hare**: a slow pointer (one step) and a fast pointer (two steps). If there is a loop, the fast pointer laps the slow one and they meet inside the loop; if there is no loop, the fast pointer reaches null.

Why the two pointers must meet

Inside a loop, each iteration the fast pointer gains one node on the slow pointer, so the gap shrinks by one every step and eventually hits zero – they collide. If the list is acyclic, the fast pointer falls off the end (reaches null) first. This detects a cycle in one pass with only two pointers, unlike a hash-set approach that needs $O(n)$ memory.

Gaining One Node per Step Guarantees a Meeting

Relative to the slow pointer, the fast pointer moves one node closer each iteration. On a finite cycle the distance between them is bounded, so it must reach zero – a collision – proving a loop exists.

Algorithm

1. `slow = fast = head.`

2. While **fast** and **fast->next**: advance **slow** by one, **fast** by two; if they are equal, return true.
3. If the loop exits, return false.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ (node 4 links back to 2).

- slow/fast advance; fast laps and meets slow at some node inside {2,3,4}.
- Meeting confirms a cycle – return true.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 bool hasCycle(ListNode* head) {
10     ListNode* slow = head;
11     ListNode* fast = head;
12     while (fast && fast->next) {
13         slow = slow->next;
14         fast = fast->next->next;
15         if (slow == fast) return true;           // collision inside
16         loop
17     }
18     return false;
19 }
20
21 int main() {
22     ListNode* a = new ListNode(1);
23     a->next = new ListNode(2);
24     a->next->next = new ListNode(3);
25     a->next->next->next = new ListNode(4);
26     a->next->next->next->next = a->next;           // 4 -> 2, a cycle
27     cout << (hasCycle(a) ? "cycle" : "no cycle") << "\n";
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$ (a hash set would cost $O(n)$).

Key Takeaway

Floyd's cycle detection uses two speeds and constant memory; a collision proves a loop. The same meeting point locates the loop's start and length (next two sections), so this is the gateway to all cycle problems.

1.9 Starting Point of a Loop

Problem Statement. If a linked list has a cycle, return the node where the cycle begins; otherwise return null. Use $O(1)$ extra space.

Pattern Recognition

After Floyd's slow/fast pointers meet inside the loop, a beautiful fact kicks in: **resetting one pointer to the head and advancing both one step at a time, they meet exactly at the loop's start.** It follows from the distance arithmetic of the tortoise-hare.

The distance argument

Let the distance from head to the loop start be L , and let the slow and fast pointers meet k nodes into the loop. When they meet, the fast pointer has travelled twice the slow pointer's distance; working through the algebra shows that L equals the remaining distance from the meeting point around to the loop start. So if you place one pointer at the head and leave the other at the meeting point, and move both one step at a time, they converge precisely at the loop's entrance.

Head to Start Equals Meeting Point to Start

The tortoise-hare meeting point is positioned so that the number of steps from the head to the loop start equals the number of steps from the meeting point (going forward around the loop) to the loop start. Two synchronized one-step walks therefore collide at the start.

Algorithm

1. Run slow/fast until they meet; if fast reaches null, return null (no loop).
2. Move `slow` to the head. Advance `slow` and `fast` one step each until they are equal.
3. That node is the loop's start.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ (loop starts at 2).

- Slow/fast meet inside the loop; reset slow to head (1).
- One step each: they converge at node 2 – the loop start.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* detectCycle(ListNode* head) {
10     ListNode* slow = head;
11     ListNode* fast = head;
```

```

12     while (fast && fast->next) {
13         slow = slow->next;
14         fast = fast->next->next;
15         if (slow == fast) {                                     // meeting point
16             slow = head;
17             while (slow != fast) { slow = slow->next; fast = fast->next
18                 ; }
19             return slow;                                       // loop start
20         }
21     }
22     return nullptr;
23 }
24
25 int main() {
26     ListNode* a = new ListNode(1);
27     a->next = new ListNode(2);
28     a->next->next = new ListNode(3);
29     a->next->next->next = new ListNode(4);
30     a->next->next->next->next = a->next;                         // 4 -> 2
31     ListNode* start = detectCycle(a);
32     cout << (start ? start->val : -1) << "\n";                // 2
33 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

The loop's start is found by resetting one pointer to the head after the tortoise-hare collision and walking both one step at a time. The underlying “ L equals meeting-to-start” identity is worth remembering – it is asked as a follow-up to cycle detection constantly.

1.10 Length of a Loop

Problem Statement. If a linked list contains a cycle, return the number of nodes in the cycle; otherwise return 0.

Pattern Recognition

Once Floyd's pointers meet inside the loop, **keep one pointer fixed at the meeting node and walk the other around until it returns** – the number of steps is the loop length. The collision guarantees both pointers are on the cycle.

Walk once around from the meeting point

Run the tortoise-hare until `slow` and `fast` coincide; that node lies on the cycle. Now advance a single pointer from that node, counting steps, until it comes back to the same node. Since it started on the cycle, it traverses exactly the cycle once, and the count is the loop's length. If the pointers never meet (fast reaches null), the length is 0.

A Full Lap from Any Cycle Node Measures It

Any node known to be on the cycle can measure the cycle's size: leave it in place and count steps until a moving pointer returns to it. The Floyd meeting point conveniently provides such a node.

Algorithm

1. Run slow/fast until they meet; if no meeting, return 0.
2. From the meeting node, advance a pointer counting steps until it returns to the meeting node.
3. Return the count.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ (loop $2 \rightarrow 3 \rightarrow 4 \rightarrow 2$, length 3).

- Pointers meet inside $\{2, 3, 4\}$. From the meeting node, count a lap: 3 steps back to it.
- Length = 3.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val; ListNode* next;
6      ListNode(int v) : val(v), next(nullptr) {}
7  };
8
9  int loopLength(ListNode* head) {
10     ListNode* slow = head;
11     ListNode* fast = head;
12     while (fast && fast->next) {
13         slow = slow->next;
14         fast = fast->next->next;
15         if (slow == fast) { // on the cycle
16             int len = 1;
17             ListNode* p = slow->next;
18             while (p != slow) { p = p->next; len++; }
19             return len;
20         }
21     }
22     return 0;
23 }
24
25 int main() {
26     ListNode* a = new ListNode(1);
27     a->next = new ListNode(2);
28     a->next->next = new ListNode(3);
29     a->next->next->next = new ListNode(4);
30     a->next->next->next->next = a->next; // 4 -> 2
31     cout << loopLength(a) << "\n"; // 3
32     return 0;

```

33 }

Complexity Analysis**Time Complexity.** $O(n)$. **Space Complexity.** $O(1)$.**Key Takeaway**

Loop length is one lap counted from the tortoise-hare meeting node. Cycle detection, its start, and its length are three questions answered by the same slow/fast collision – keep the trio together in memory.

1.11 Check if a Linked List is a Palindrome

Problem Statement. Given the head of a singly linked list, determine whether the sequence of values reads the same forwards and backwards, using $O(1)$ extra space.

Pattern Recognition

Without random access, comparing front to back means **finding the middle (slow/fast), reversing the second half, then walking both halves in step**. Equality of the two halves proves a palindrome. This combines two earlier techniques into one.

Middle, reverse, compare

Use the tortoise-hare to reach the middle. Reverse the second half in place (the iterative reversal). Now walk one pointer from the head and another from the reversed second half, comparing values; if all match, the list is a palindrome. Optionally restore the list by reversing the second half back. This achieves $O(1)$ space, versus copying values into an array ($O(n)$).

Reverse Half to Compare Without Extra Memory

A palindrome check needs to read the list backward, which a singly linked list cannot do – so reverse the back half physically. Comparing the untouched front half against the reversed back half is exactly the front-to-back comparison, in constant space.

Algorithm

1. Find the middle with slow/fast.
2. Reverse the second half.
3. Compare the first half and the reversed second half node by node.
4. Return whether all comparisons matched.

Dry run

Dry Run. List $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$.

- Middle after node 2 (second); reverse back half $\rightarrow 1 \rightarrow 2$.
- Compare $1=1$, $2=2$ – palindrome.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* reverse(ListNode* head) {
16     ListNode* prev = nullptr;
17     while (head) { ListNode* nx = head->next; head->next = prev; prev =
18         head; head = nx; }
19     return prev;
20 }
21
22 bool isPalindrome(ListNode* head) {
23     ListNode* slow = head;
24     ListNode* fast = head;
25     while (fast && fast->next) { slow = slow->next; fast = fast->next->
26         next; }
27     ListNode* second = reverse(slow); // reverse back half
28
29     ListNode* p1 = head;
30     ListNode* p2 = second;
31     while (p2) {
32         if (p1->val != p2->val) return false;
33         p1 = p1->next; p2 = p2->next;
34     }
35     return true;
36 }
37
38 int main() {
39     cout << isPalindrome(build({1, 2, 2, 1})) << "\n"; // 1
40     cout << isPalindrome(build({1, 2, 3})) << "\n"; // 0
41     return 0;
42 }

```

Complexity Analysis

Time Complexity. $O(n)$ – find middle, reverse, compare, each linear. **Space Complexity.** $O(1)$.

Key Takeaway

Palindrome checking composes slow/fast (middle) with in-place reversal (back half) to compare front and back in constant space. It is the classic demonstration that combining two basic list techniques beats the $O(n)$ - memory array copy.

1.12 Segregate Odd and Even Position Nodes

Problem Statement. Given a linked list, group all nodes at odd positions together followed by the nodes at even positions, preserving the relative order within each group. Positions are 1-indexed; do it in place.

Pattern Recognition

Rearranging by position parity is a **two-chain weave**: maintain an odd chain and an even chain, appending alternate nodes to each, then splice the even chain onto the tail of the odd chain. Only pointers move.

Build two chains, then join

Keep an **odd** pointer starting at the head and an **even** pointer starting at the second node, plus a saved **evenHead**. Repeatedly link **odd->next = even->next** (skip the even node) and advance **odd**; then **even->next = odd->next** and advance **even**. This splits the list into odd-position and even-position chains in a single pass. Finally attach **evenHead** after the last odd node.

Interleave by Skipping Every Other Node

Advancing each chain by two positions (skipping the other parity) threads the odd and even nodes apart without moving data. The saved **evenHead** lets the two chains be reconnected at the end.

Algorithm

1. If the list has fewer than 3 nodes, return it unchanged.
2. **odd** = **head**, **even** = **head->next**, **evenHead** = **even**.
3. While **even** and **even->next**: relink **odd** then **even** by skipping one node each; advance both.
4. **odd->next** = **evenHead**; return the head.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

- Odd chain $1 \rightarrow 3 \rightarrow 5$; even chain $2 \rightarrow 4$.
- Join: $1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;

```

```

11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* oddEvenList(ListNode* head) {
16     if (!head || !head->next) return head;
17     ListNode* odd = head;
18     ListNode* even = head->next;
19     ListNode* evenHead = even;
20     while (even && even->next) {
21         odd->next = even->next;    odd = odd->next;    // splice next
22                                     odd
23         even->next = odd->next;    even = even->next;  // splice next
24                                     even
25     }
26     odd->next = evenHead;          // attach evens
27                                     after odds
28     return head;
29 }
30
31 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
32     cout << "\n"; }
33
34 int main() {
35     print(oddEvenList(build({1, 2, 3, 4, 5})));    // 1 3 5 2 4
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$.

Key Takeaway

Grouping by position parity is a two-chain weave joined at the end – a pure pointer rearrangement. The “split into interleaved chains, then reconnect” idea also underlies reordering and unzip-style list problems.

1.13 Remove the Nth Node from the End

Problem Statement. Given the head of a linked list and an integer n , remove the n -th node counting from the end and return the head, in one pass.

Pattern Recognition

“ n -th from the end in one pass” is a **two-pointer gap**: advance a lead pointer n nodes ahead, then move both until the lead reaches the end – the trailing pointer now sits just before the node to delete. A dummy head handles removing the first node cleanly.

Fix the gap, then slide it

Place a dummy node before the head. Advance a **fast** pointer n steps from the dummy. Then move **fast** and a **slow** pointer (starting at the dummy) together until **fast** reaches the last node. Because the gap between them is exactly n , **slow** ends up just before the target; unlink it with `slow->next = slow->next->next`. Return `dummy.next`.

A Fixed Gap Finds the Position from the End

The distance from the end is converted into a fixed lead of n nodes. Sliding both pointers until the lead hits the tail leaves the trailing pointer at the predecessor of the n -th-from-last node – all in a single traversal.

Algorithm

1. Dummy before head; `fast = dummy`, advance it n steps.
2. Move `fast` and `slow` (from dummy) together until `fast->next` is null.
3. `slow->next = slow->next->next`; return `dummy.next`.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $n = 2$.

- `fast` advances 2 to node 2; then both slide until `fast` at 5.
- `slow` at 3; remove $3 \rightarrow \text{next}$ (node 4). Result $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* removeNthFromEnd(ListNode* head, int n) {
16     ListNode dummy(0); dummy.next = head;
17     ListNode* fast = &dummy;
18     for (int i = 0; i < n; i++) fast = fast->next; // lead by n
19     ListNode* slow = &dummy;
20     while (fast->next) { fast = fast->next; slow = slow->next; }
21     slow->next = slow->next->next; // unlink target
22     return dummy.next;
23 }
24
25 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
26     cout << "\n"; }
27
28 int main() {
29     print(removeNthFromEnd(build({1, 2, 3, 4, 5}), 2)); // 1 2 3 5
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. $O(n)$ – a single pass. **Space Complexity.** $O(1)$.

Key Takeaway

A fixed two-pointer gap of n turns “from the end” into a one-pass position. The dummy node makes deleting the head uniform with any other node – the recurring reason to prepend a dummy.

1.14 Delete the Middle Node

Problem Statement. Given the head of a linked list, delete the middle node (for even length, the second middle) and return the head. If the list has one node, return null.

Pattern Recognition

Deleting the middle combines the slow/fast middle-finder with the need for the node *before* the middle. Advance **fast** by two but start it one step ahead (or track **prev**) so that when **fast** reaches the end, **slow** stops just before the middle, ready to unlink it.

Stop slow one node early

Run the tortoise-hare, but arrange the offsets so **slow** lands on the *predecessor* of the middle rather than the middle itself – for instance by giving **fast** a two-node head start. Then **slow->next = slow->next->next** removes the middle node. Handle the single-node list as a special case (return null).

Delete Needs the Predecessor

Finding the middle is not enough to delete it in a singly linked list; you need its predecessor to rewire **next**. Offsetting the fast pointer so **slow** halts one node early yields that predecessor directly, avoiding a second pass.

Algorithm

1. If the list has one node, return null.
2. **slow** = head, **fast** = head->next->next.
3. While **fast** and **fast->next**: advance **slow** by one, **fast** by two.
4. **slow->next** = **slow->next->next**; return the head.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (delete second middle, node 3).

- **slow**= 1, **fast**= 3. Step: **slow**= 2, **fast**=null.
- Remove $2 \rightarrow \text{next}$ (node 3). Result $1 \rightarrow 2 \rightarrow 4$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* deleteMiddle(ListNode* head) {
16     if (!head->next) return nullptr;           // single node
17     ListNode* slow = head;
18     ListNode* fast = head->next->next;         // two-node head
19     // start
20     while (fast && fast->next) {
21         slow = slow->next;
22         fast = fast->next->next;
23     }
24     slow->next = slow->next->next;             // unlink the middle
25     return head;
26 }
27
28 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
29     cout << "\n"; }
30
31 int main() {
32     print(deleteMiddle(build({1, 2, 3, 4, 5}))); // 1 2 4 5
33     print(deleteMiddle(build({1, 2, 3, 4})));    // 1 2 4
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

To delete the middle, offset the fast pointer so slow halts at the middle's predecessor – then a single rewire removes it. Adjusting the two-pointer offset to land on the predecessor is a common tweak to the middle-finder.

1.15 Sort a Linked List

Problem Statement. Given the head of a linked list, sort it in ascending order and return the new head, ideally in $O(n \log n)$ time and $O(1)$ auxiliary space.

Pattern Recognition

Merge sort is the natural sort for linked lists: splitting by the middle (slow/fast) and merging two sorted lists are both cheap pointer operations, and unlike arrays no random access is needed. Quicksort's partition is awkward on lists, so merge sort is the standard choice.

Split, sort halves, merge

Find the middle with the tortoise-hare and cut the list into two halves. Recursively sort each half. Then merge the two sorted halves by repeatedly appending the smaller current node – exactly the merge step of array merge sort, but relinking nodes instead of copying values. The recursion bottoms out at single-node (already sorted) lists.

Lists Merge and Split Without Extra Arrays

Splitting a list is just severing it at the middle; merging is relinking nodes in order. Both are $O(1)$ -space pointer work, so merge sort on a list needs no auxiliary array – its recursion stack is the only extra space.

Algorithm

1. If the list has 0 or 1 nodes, return it.
2. Split at the middle into two halves.
3. Recursively sort both halves; merge them into one sorted list.

Dry run

Dry Run. $3 \rightarrow 1 \rightarrow 2$.

- Split into $3 \rightarrow 1$ and 2 . Sort $3 \rightarrow 1$ into $1 \rightarrow 3$.
- Merge $1 \rightarrow 3$ with 2 : $1 \rightarrow 2 \rightarrow 3$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* merge(ListNode* a, ListNode* b) {
16     ListNode dummy(0), *t = &dummy;
17     while (a && b) {
18         if (a->val <= b->val) { t->next = a; a = a->next; }

```

```

19         else { t->next = b; b = b->next; }
20         t = t->next;
21     }
22     t->next = a ? a : b; // attach the
23     remainder
24     return dummy.next;
25 }
26
27 ListNode* sortList(ListNode* head) {
28     if (!head || !head->next) return head;
29     ListNode* slow = head; ListNode* fast = head->next;
30     while (fast && fast->next) { slow = slow->next; fast = fast->next->
31         next; }
32     ListNode* mid = slow->next;
33     slow->next = nullptr; // cut into two
34     halves
35     return merge(sortList(head), sortList(mid));
36 }
37
38 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
39     cout << "\n"; }
40
41 int main() {
42     print(sortList(build({4, 2, 1, 3}))); // 1 2 3 4
43     return 0;
44 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ – $\log n$ split levels, $O(n)$ merge per level. **Space Complexity.** $O(\log n)$ recursion stack.

Key Takeaway

Merge sort suits linked lists because split and merge are pointer operations needing no random access or extra array. It is the default answer to “sort a list” and reuses the middle-finder and merge routines seen elsewhere.

1.16 Sort a Linked List of 0s, 1s, and 2s

Problem Statement. Given a linked list whose values are only 0, 1, or 2, sort it in a single pass without swapping data, returning the head.

Pattern Recognition

With just three distinct values, sorting is a **three-chain partition**: build separate chains for 0s, 1s, and 2s in one pass, then concatenate them. No comparisons or $O(n \log n)$ sort are needed – this is the linked-list Dutch-national-flag.

Three dummy heads, then join

Create three dummy-headed chains, one per value. Walk the list once, appending each node to the chain matching its value. Then link the 0-chain to the 1-chain to the 2-chain, terminating the last with null. Because you distribute nodes by value and reassemble in order, the result is sorted. Using dummy heads avoids special-casing empty chains.

Bucket by Value, Then Concatenate

When the number of distinct keys is a small constant, a counting/bucketing pass beats comparison sorting. Three chains bucket the nodes; concatenation orders them – linear time, no data movement, just relinking.

Algorithm

1. Three dummy-headed chains for 0, 1, 2; keep their tails.
2. Walk the list, appending each node to its value's chain.
3. Join $0 \rightarrow 1 \rightarrow 2$ (skipping empty chains via the dummy links); null- terminate; return the combined head.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 0$.

- Chains: $0s = 0 \rightarrow 0$, $1s = 1 \rightarrow 1$, $2s = 2 \rightarrow 2$.
- Join: $0 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 2$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val; ListNode* next;
6      ListNode(int v) : val(v), next(nullptr) {}
7  };
8
9  ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* sortColors(ListNode* head) {
16     ListNode d0(0), d1(0), d2(0);           // dummy heads
17     ListNode *t0 = &d0, *t1 = &d1, *t2 = &d2;
18     for (ListNode* c = head; c; c = c->next) {
19         if (c->val == 0) { t0->next = c; t0 = c; }
20         else if (c->val == 1) { t1->next = c; t1 = c; }
21         else { t2->next = c; t2 = c; }
22     }
23     t2->next = nullptr;                       // terminate
24     t1->next = d2.next;                       // join 1s -> 2s
25     t0->next = d1.next ? d1.next : d2.next;   // join 0s -> (1s or
26     return d0.next;                          2s)
27 }
28
29 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
30     cout << "\n"; }
```

```

30
31 int main() {
32     print(sortColors(build({1, 2, 0, 1, 2, 0}))); // 0 0 1 1 2 2
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$ (three dummy nodes).

Key Takeaway

A constant number of distinct values makes bucketing beat comparison sorting: three chains, one pass, then concatenate. The dummy-headed chain pattern – collect into groups, then splice – reappears in partitioning and stable-reordering problems.

1.17 Intersection Point of Two Linked Lists

Problem Statement. Given the heads of two singly linked lists that may merge at some node (forming a Y shape), return the first common node, or null if they never intersect. Use $O(1)$ extra space.

Pattern Recognition

The elegant solution is the **two-pointer switch**: walk one pointer along list A then list B, and another along list B then list A. They travel the same total distance, so they arrive at the intersection simultaneously (or at null together).

Equalise the path lengths automatically

Let the lists have lengths a and b with a shared tail of length c . Pointer 1 walks a then b ; pointer 2 walks b then a ; both cover $a + b$ nodes. After the switch, each is exactly c nodes from the end, so they meet at the first shared node. If there is no intersection, both reach null at the same time. This removes the need to measure and align lengths manually.

Switching Lists Cancels the Length Difference

By having each pointer traverse both lists, the head-start difference $|a - b|$ is absorbed: after the switch they are equidistant from the merge point. The first place they coincide is the intersection.

Algorithm

1. $p = \text{headA}$, $q = \text{headB}$.
2. Advance both; when p hits null, redirect it to headB ; when q hits null, redirect to headA .
3. Stop when $p == q$; that node (or null) is the answer.

Dry run

Dry Run. $A = 1 \rightarrow 9 \rightarrow 7$, $B = 6 \rightarrow 7$, sharing the node 7.

- p : 1, 9, 7, (B)6, 7. q : 6, 7, (A)1, 9, 7.
- Both reach the shared 7 at the same step – the intersection.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* getIntersectionNode(ListNode* a, ListNode* b) {
10     if (!a || !b) return nullptr;
11     ListNode* p = a; ListNode* q = b;
12     while (p != q) {
13         p = p ? p->next : b;           // switch to B at the
14         q = q ? q->next : a;           // switch to A at the
15     }
16     return p;                           // meeting node or
17 }                                     null
18
19 int main() {
20     ListNode* shared = new ListNode(7);
21     shared->next = new ListNode(8);
22     ListNode* a = new ListNode(1);
23     a->next = new ListNode(9); a->next->next = shared;
24     ListNode* b = new ListNode(6); b->next = shared;
25
26     ListNode* r = getIntersectionNode(a, b);
27     cout << (r ? r->val : -1) << "\n"; // 7
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(a + b)$ – each pointer walks both lists once. **Space Complexity.** $O(1)$ (a hash set would cost $O(n)$).

Key Takeaway

The two-pointer list switch equalises unequal lengths automatically and finds the intersection in constant space. It is a favourite interview trick precisely because the length-cancellation is so clean.

1.18 Add One to a Number Represented by a Linked List

Problem Statement. A non-negative integer is stored as a linked list, most significant digit first (head). Add one to the number and return the head of the resulting list.

Pattern Recognition

The carry propagates from the *least* significant digit, but the list is stored most-significant first – so either **reverse**, **add**, **reverse back**, or use recursion to reach the last node and carry upward on the return trip. Both avoid needing backward pointers.

Carry from the back via recursion

Recurse to the tail; the deepest call adds one and returns any carry. Each frame adds the incoming carry to its digit, updates the digit modulo 10, and returns the new carry (1 if the digit was 9, else 0). If the head still produces a carry after processing, prepend a new leading 1 node. This treats the recursion stack as the “reversed” traversal, so no explicit reversal is required.

The Return Trip Is the Least-Significant-First Pass

Recursion reaches the last (least significant) digit first on the way down, then processes digits back-to-front on the way up – exactly the order addition needs. A leftover carry at the very top means a new most- significant digit.

Algorithm

1. Recurse to the last node; add 1 there, return the carry.
2. Each frame: $\text{sum} = \text{digit} + \text{carry}$; set digit to $\text{sum} \% 10$; return $\text{sum} / 10$.
3. If the top-level carry is 1, prepend a new node with value 1.

Dry run

Dry Run. List $1 \rightarrow 2 \rightarrow 9$ (number 129).

- Last digit $9 + 1 = 10$: digit 0, carry 1. Next $2 + 1 = 3$, carry 0.
- Head 1 unchanged. Result $1 \rightarrow 3 \rightarrow 0$ (130).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val; ListNode* next;
6      ListNode(int v) : val(v), next(nullptr) {}
7  };
8
9  ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 int addHelper(ListNode* node) {
16     if (!node) return 1; // seed the +1 at the
17                         // end
18     int carry = addHelper(node->next);
19     int sum = node->val + carry;
20     node->val = sum % 10;
21     return sum / 10;
22 }
23
24 ListNode* addOne(ListNode* head) {
25     int carry = addHelper(head);

```

```

25     if (carry) {                                     // new leading digit
26         ListNode* node = new ListNode(carry);
27         node->next = head;
28         return node;
29     }
30     return head;
31 }
32
33 void print(ListNode* h) { for (; h; h = h->next) cout << h->val; cout
    << "\n"; }
34
35 int main() {
36     print(addOne(build({1, 2, 9})));                // 130
37     print(addOne(build({9, 9, 9})));                // 1000
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ recursion stack (or $O(1)$ with the reverse-add-reverse approach).

Key Takeaway

Arithmetic on a most-significant-first list needs a back-to-front pass, supplied for free by recursion's return trip (or by reversing). A leftover carry at the head becomes a new leading digit – the one edge case to remember.

1.19 Add Two Numbers Represented by Linked Lists

Problem Statement. Two non-negative integers are stored as linked lists with the *least* significant digit first. Add them and return the sum as a linked list in the same order.

Pattern Recognition

Because the digits are stored least-significant first, addition proceeds head-to-tail with a running **carry** – a straightforward simultaneous walk of both lists, building the result node by node, exactly like grade-school addition.

Walk both lists with a carry

Traverse both lists together. At each step add the two current digits (or 0 if a list is exhausted) plus the carry; the new digit is the sum modulo 10 and the new carry is the sum divided by 10. Append the digit to the result via a dummy-headed builder. Continue until both lists end and the carry is zero – a final carry produces one more leading node.

Least-Significant-First Makes Addition a Forward Walk

Storing digits reversed means the carry flows in the same direction as traversal, so no reversal or recursion is needed – just a single forward pass summing aligned digits with a carry, the most natural form of the problem.

Algorithm

1. Dummy-headed result; **carry** = 0.

2. While either list has nodes or `carry` is nonzero: sum the available digits and carry; append `sum % 10`; set `carry = sum / 10`.
3. Return `dummy.next`.

Dry run

Dry Run. $2 \rightarrow 4 \rightarrow 3$ (342) and $5 \rightarrow 6 \rightarrow 4$ (465).

- $2+5=7$; $4+6=10 \rightarrow 0$ carry 1; $3+4+1=8$.
- Result $7 \rightarrow 0 \rightarrow 8$ (807).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* addTwoNumbers(ListNode* a, ListNode* b) {
16     ListNode dummy(0), *t = &dummy;
17     int carry = 0;
18     while (a || b || carry) {
19         int sum = carry;
20         if (a) { sum += a->val; a = a->next; }
21         if (b) { sum += b->val; b = b->next; }
22         carry = sum / 10;
23         t->next = new ListNode(sum % 10);
24         t = t->next;
25     }
26     return dummy.next;
27 }
28
29 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
30     cout << "\n"; }
31
32 int main() {
33     print(addTwoNumbers(build({2, 4, 3}), build({5, 6, 4}))); // 7 0
34     return 0;
35 }
```

Complexity Analysis

Time Complexity. $O(\max(a, b))$ – one pass over the longer list. **Space Complexity.** $O(\max(a, b))$ for the result list.

Key Takeaway

Least-significant-first storage turns list addition into a single forward carry walk – the cleanest arithmetic setup. Contrast with Add One’s most-significant-first order, which needs recursion or reversal to reach the low digit first.

1.20 Delete All Occurrences of a Key in a Doubly Linked List

Problem Statement. Given a doubly linked list and a key, delete every node whose value equals the key, returning the (possibly new) head.

Pattern Recognition

The DLL’s **prev** pointer makes this a single-pass sweep: at each matching node, splice its neighbours together in $O(1)$ using both links, handling head and tail as the only edge cases.

Splice out each match in place

Walk the list. When a node’s value equals the key, connect its predecessor’s **next** to its successor and its successor’s **prev** to its predecessor. If the deleted node was the head, advance the head. Because a DLL node knows both neighbours, each deletion is constant time with no re-walk – the exact advantage over a singly linked list.

Both Links Make Deletion Local

Removing a node requires rewiring the links on *both* sides. The **prev** pointer supplies the left neighbour instantly, so an arbitrary node – including several in one pass – is removed in $O(1)$ each.

Algorithm

1. Walk from the head. For each node equal to the key: rewire **prev** and **next** of its neighbours; if it was the head, move the head forward.
2. Continue to the next node.
3. Return the head.

Dry run

Dry Run. DLL $1 \leftrightarrow 2 \leftrightarrow 2 \leftrightarrow 3$, key 2.

- Delete both 2s by splicing neighbours.
- Result $1 \leftrightarrow 3$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int val; Node* prev; Node* next;
6     Node(int v) : val(v), prev(nullptr), next(nullptr) {}

```

```

7 };
8
9 Node* build(const vector<int>& a) {
10     Node dummy(0), *t = &dummy;
11     for (int x : a) { Node* n = new Node(x); t->next = n; n->prev = t;
12         t = n; }
13     Node* head = dummy.next; if (head) head->prev = nullptr;
14     return head;
15 }
16
17 Node* deleteAll(Node* head, int key) {
18     Node* cur = head;
19     while (cur) {
20         Node* nxt = cur->next;
21         if (cur->val == key) {
22             if (cur->prev) cur->prev->next = cur->next;
23             else head = cur->next; // deleting the head
24             if (cur->next) cur->next->prev = cur->prev;
25         }
26         cur = nxt;
27     }
28     return head;
29 }
30
31 void print(Node* h) { for (; h; h = h->next) cout << h->val << " ";
32     cout << "\n"; }
33
34 int main() {
35     print(deleteAll(build({1, 2, 2, 3, 2}), 2)); // 1 3
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$.

Key Takeaway

Deleting all matches in a DLL is a single-pass splice using both links, with head/tail as the only special cases. This is the everyday DLL deletion skill that LRU/LFU caches rely on.

1.21 Find Pairs with a Given Sum in a Sorted DLL

Problem Statement. Given a doubly linked list sorted in ascending order and a target, find all pairs of nodes whose values sum to the target, using $O(1)$ extra space.

Pattern Recognition

A sorted DLL supports the array **two-pointer** technique directly: one pointer at the head (smallest), one at the tail (largest). The **prev** pointer lets the right pointer move backward, which a singly linked list cannot do.

Converge from both ends

Place **left** at the head and **right** at the tail. Compare their sum to the target: if equal, record the pair and move both inward; if too small, advance **left** (via **next**) to increase the sum; if too

large, retreat `right` (via `prev`) to decrease it. Stop when the pointers meet or cross. Sortedness guarantees this monotonic convergence finds every pair without revisiting.

The prev Pointer Enables Backward Two Pointers

The classic two-sum-on-sorted-array needs a right pointer that moves left. A DLL's `prev` provides exactly that, so the linear two-pointer sweep transfers unchanged from arrays to sorted doubly linked lists.

Algorithm

1. `left = head; right = last node.`
2. While `left != right` and not crossed: compare `left->val + right->val` to the target; record on equality and move both, else move the appropriate pointer.
3. Output all recorded pairs.

Dry run

Dry Run. Sorted DLL 1, 2, 3, 4, 6; target 6.

- (1, 6)? $7 > 6$, move right to 4. (1, 4)? $5 < 6$, move left to 2. (2, 4) = 6 – record.
- Move both; pointers cross. Pair found: (2, 4).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int val; Node* prev; Node* next;
6     Node(int v) : val(v), prev(nullptr), next(nullptr) {}
7 };
8
9 Node* build(const vector<int>& a) {
10     Node dummy(0), *t = &dummy;
11     for (int x : a) { Node* n = new Node(x); t->next = n; n->prev = t;
12         t = n; }
13     Node* head = dummy.next; if (head) head->prev = nullptr;
14     return head;
15 }
16
17 vector<pair<int,int>> pairsWithSum(Node* head, int target) {
18     vector<pair<int,int>> res;
19     if (!head) return res;
20     Node* left = head;
21     Node* right = head;
22     while (right->next) right = right->next; // go to tail
23     while (left != right && right->next != left) {
24         int sum = left->val + right->val;
25         if (sum == target) {
26             res.push_back({left->val, right->val});
27             left = left->next; right = right->prev;
28         } else if (sum < target) left = left->next;
29         else right = right->prev;
30     }
31     return res;
32 }

```

```

29     }
30     return res;
31 }
32
33 int main() {
34     for (auto& [a, b] : pairsWithSum(build({1, 2, 3, 4, 6}), 6))
35         cout << "(" << a << "," << b << ")" << " ";
36     cout << "\n"; // (2,4)
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n)$ after reaching the tail (also $O(n)$). **Space Complexity.** $O(1)$ beyond the output.

Key Takeaway

Sorted-DLL pair-sum is the two-pointer two-sum with the right pointer moving backward via `prev`. The DLL's backward link is precisely what lets this array technique work on a list.

1.22 Remove Duplicates from a Sorted DLL

Problem Statement. Given a doubly linked list sorted in ascending order, remove nodes with duplicate values so each value appears once, returning the head.

Pattern Recognition

In a sorted list, duplicates are **adjacent**, so a single pass comparing each node to the next – and splicing out equal successors – removes all duplicates. The `prev` pointer keeps the deletions $O(1)$.

Skip equal neighbours

Walk from the head. While the current node's value equals its successor's, unlink the successor (rewire the current node's `next` and the node-after's `prev`). Only advance to the next distinct value once no more equal successors remain. Sortedness guarantees all copies of a value are consecutive, so this local check suffices.

Sorted Means Duplicates Are Adjacent

Because equal values sit next to each other in a sorted list, comparing only neighbours catches every duplicate – no hashing or full scan needed. The DLL links make each removal constant time.

Algorithm

1. `cur = head`.
2. While `cur` and `cur->next`: if equal, splice out `cur->next`; else advance `cur`.
3. Return the head.

Dry run

Dry Run. Sorted DLL 1,1,2,3,3.

- At first 1: remove the second 1. At 2: no dup. At first 3: remove the second 3.
- Result 1,2,3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int val; Node* prev; Node* next;
6     Node(int v) : val(v), prev(nullptr), next(nullptr) {}
7 };
8
9 Node* build(const vector<int>& a) {
10     Node dummy(0), *t = &dummy;
11     for (int x : a) { Node* n = new Node(x); t->next = n; n->prev = t;
12         t = n; }
13     Node* head = dummy.next; if (head) head->prev = nullptr;
14     return head;
15 }
16
17 Node* removeDuplicates(Node* head) {
18     Node* cur = head;
19     while (cur && cur->next) {
20         if (cur->val == cur->next->val) {
21             Node* dup = cur->next;
22             cur->next = dup->next;           // splice out
23             duplicate
24             if (dup->next) dup->next->prev = cur;
25         } else {
26             cur = cur->next;
27         }
28     }
29     return head;
30 }
31
32 void print(Node* h) { for (; h; h = h->next) cout << h->val << " ";
33     cout << "\n"; }
34
35 int main() {
36     print(removeDuplicates(build({1, 1, 2, 3, 3}))); // 1 2 3
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass. **Space Complexity.** $O(1)$.

Key Takeaway

On a sorted list, duplicates are neighbours, so a neighbour-comparison pass removes them all. Recognising the sorted invariant turns a potential hashing problem into an $O(1)$ -space local sweep.

1.23 Reverse Nodes in Groups of K

Problem Statement. Given a linked list and an integer k , reverse the nodes in consecutive groups of k . If the final group has fewer than k nodes, leave it as is. Return the head.

Pattern Recognition

This is the iterative reversal applied **block by block**: for each group of k nodes, check that k nodes remain, reverse that block in place, and stitch it between the already-processed part and the rest. A dummy node anchors the stitching.

Count, reverse a block, reconnect

Maintain a pointer to the node before the current group. First verify k nodes remain (walk ahead k steps); if not, stop. Reverse the k nodes with the standard save–flip–advance loop. Then reconnect: the previous group’s tail points to the new (reversed) block head, and the block’s tail points to the following node. Move the “previous group tail” pointer to the block’s tail and repeat.

Reverse in Place, Then Re-stitch the Seams

The only complexity beyond plain reversal is bookkeeping at the two seams of each block – linking the prior tail to the new head and the new tail to the remainder. A dummy node removes the special case for the very first block.

Algorithm

1. Dummy before head; `groupPrev = dummy`.
2. Loop: check k nodes remain; if not, break. Reverse the next k nodes; reconnect the seams; advance `groupPrev` to the block’s (new) tail.
3. Return `dummy.next`.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $k = 2$.

- Reverse $[1, 2] \rightarrow 2 \rightarrow 1$; reverse $[3, 4] \rightarrow 4 \rightarrow 3$; $[5]$ left as is.
- Result $2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ListNode {
5     int val; ListNode* next;
6     ListNode(int v) : val(v), next(nullptr) {}
7 };
8
9 ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;

```

```

13 }
14
15 ListNode* reverseKGroup(ListNode* head, int k) {
16     ListNode dummy(0); dummy.next = head;
17     ListNode* groupPrev = &dummy;
18
19     while (true) {
20         ListNode* kth = groupPrev;
21         for (int i = 0; i < k && kth; i++) kth = kth->next; // k-th
22         // node
23         if (!kth) break; // fewer
24         // than k left
25
26         ListNode* groupNext = kth->next;
27         ListNode* prev = groupNext;
28         ListNode* curr = groupPrev->next;
29         while (curr != groupNext) { //
30             // reverse the block
31             ListNode* nx = curr->next;
32             curr->next = prev;
33             prev = curr;
34             curr = nx;
35         }
36         ListNode* newTail = groupPrev->next; // old
37         // head -> new tail
38         groupPrev->next = kth; //
39         // stitch to new head
40         groupPrev = newTail;
41     }
42     return dummy.next;
43 }
44
45 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
46     cout << "\n"; }
47
48 int main() {
49     print(reverseKGroup(build({1, 2, 3, 4, 5}), 2)); // 2 1 4 3 5
50     return 0;
51 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is reversed once. **Space Complexity.** $O(1)$.

Key Takeaway

Reverse-in- k -groups is block-wise iterative reversal with careful seam stitching. The “check k remain, reverse, reconnect” loop is the hard- problem culmination of the plain reversal skill built earlier.

1.24 Rotate a Linked List

Problem Statement. Given the head of a linked list and an integer k , rotate the list to the right by k places and return the new head.

Pattern Recognition

Rotating right by k is a **relink**, not a shift: make the list circular, then break it at the right spot. Since rotating by the length gives the same list, first reduce k modulo the length.

Circularise, then cut

Walk to the tail, counting the length n , and connect the tail to the head to form a ring. Reduce k to $k \bmod n$. The new tail is the node at position $n - k$ from the start; the node after it is the new head. Walk to that new tail, set its **next** to null (breaking the ring), and return the new head. This performs any rotation with a single relink, avoiding k separate moves.

Rotation Is One Cut on a Ring

Joining the list into a circle makes every rotation just a choice of where to break it. Reducing k modulo n handles $k \geq n$, and the break point $n - k$ places the last k nodes at the front.

Algorithm

1. Find the length n and the tail; connect tail to head (ring).
2. $k \leftarrow k \bmod n$; the new tail is $n - k$ steps from the head.
3. Break after the new tail; return the node following it.

Dry run

Dry Run. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $k = 2$.

- $n = 5$, $k = 2$; new tail at position $5 - 2 = 3$ (node 3).
- Break after 3; new head 4. Result $4 \rightarrow 5 \rightarrow 1 \rightarrow 2 \rightarrow 3$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val; ListNode* next;
6      ListNode(int v) : val(v), next(nullptr) {}
7  };
8
9  ListNode* build(const vector<int>& a) {
10     ListNode dummy(0), *t = &dummy;
11     for (int x : a) { t->next = new ListNode(x); t = t->next; }
12     return dummy.next;
13 }
14
15 ListNode* rotateRight(ListNode* head, int k) {
16     if (!head || !head->next || k == 0) return head;
17     int n = 1;
18     ListNode* tail = head;
19     while (tail->next) { tail = tail->next; n++; }
20     tail->next = head; // make it circular
21 }
```

```

22     k %= n;
23     int stepsToNewTail = n - k;
24     ListNode* newTail = head;
25     for (int i = 1; i < stepsToNewTail; i++) newTail = newTail->next;
26     ListNode* newHead = newTail->next;
27     newTail->next = nullptr;           // break the ring
28     return newHead;
29 }
30
31 void print(ListNode* h) { for (; h; h = h->next) cout << h->val << " ";
    cout << "\n"; }
32
33 int main() {
34     print(rotateRight(build({1, 2, 3, 4, 5}), 2)); // 4 5 1 2 3
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass to measure, one to reach the new tail. **Space Complexity.** $O(1)$.

Key Takeaway

Rotation is circularise-then-cut: one relink instead of k moves, with $k \bmod n$ collapsing large rotations. Turning a list into a ring to simplify positional surgery is a handy general trick.

1.25 Flatten a Linked List

Problem Statement. Each node of a linked list has a **next** pointer (to the next sub-list head) and a **bottom** pointer heading a sorted vertical sub-list. Flatten the whole structure into a single sorted list linked entirely by **bottom** pointers.

Pattern Recognition

Every vertical sub-list is already sorted, so flattening is **repeated merging**: merge two sorted **bottom**-lists at a time, folding the horizontal **next** chain into one long sorted vertical list. It is the merge routine of merge sort applied across sub-lists.

Merge sub-lists right to left

Recurse along **next** to flatten everything to the right of the head first, then merge the head's vertical list with that flattened remainder. The merge compares **bottom** values and links the smaller each time, producing one sorted **bottom**-chain. Processing right-to-left means each merge combines the current sub-list with an already-fully-flattened tail.

Fold the Horizontal Lists into One Vertical Merge

Because each vertical list is sorted, the whole grid flattens by pairwise merging along the **next** direction. The result uses only **bottom** pointers, discarding the horizontal structure.

Algorithm

1. If head or head->next is null, return head.

2. Recursively flatten `head->next`.
3. Merge `head`'s vertical list with the flattened remainder by `bottom` value; return the merged `head`.

Dry run

Dry Run. Two sub-lists: $3 \downarrow 7$ and $1 \downarrow 4$ (down = bottom), linked by `next`.

- Flatten the second: $1 \rightarrow 4$. Merge with $3 \rightarrow 7$.
- Result (bottom-linked) $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Node {
5      int val;
6      Node* next;           // horizontal
7      Node* bottom;         // vertical (sorted)
8      Node(int v) : val(v), next(nullptr), bottom(nullptr) {}
9  };
10
11 Node* merge(Node* a, Node* b) {           // merge two bottom-
12     lists
13     Node dummy(0), *t = &dummy;
14     while (a && b) {
15         if (a->val <= b->val) { t->bottom = a; a = a->bottom; }
16         else { t->bottom = b; b = b->bottom; }
17         t = t->bottom;
18     }
19     t->bottom = a ? a : b;
20     return dummy.bottom;
21 }
22
23 Node* flatten(Node* head) {
24     if (!head || !head->next) return head;
25     head->next = flatten(head->next);    // flatten the rest
26     first
27     head = merge(head, head->next);
28     return head;
29 }
30
31 int main() {
32     Node* a = new Node(3); a->bottom = new Node(7);
33     Node* b = new Node(1); b->bottom = new Node(4);
34     a->next = b;
35     Node* f = flatten(a);
36     for (Node* c = f; c; c = c->bottom) cout << c->val << " ";
37     cout << "\n";           // 1 3 4 7
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(N)$ total merge work across all N nodes (with $O(N \cdot m)$ in the naive per-merge bound, m sub-lists). **Space Complexity.** $O(m)$ recursion over the `next` chain.

Key Takeaway

Flattening sorted sub-lists is repeated merging along the horizontal direction, reusing the merge-sort combine step. Recognising “already sorted pieces $\Rightarrow$ merge, don’t re-sort” is the key insight; a priority queue over sub-list heads is an alternative.

1.26 Clone a Linked List with Random Pointers

Problem Statement. Each node has a `next` pointer and a `random` pointer to any node (or null). Produce a deep copy of the list – new nodes with correctly wired `next` and `random` pointers – in $O(1)$ extra space.

Pattern Recognition

The clever $O(1)$ -space method **interleaves** each copy right after its original. Then a copy’s `random` is just the original’s `random’s next`. A final pass unweaves the two lists apart.

Interleave, wire randoms, unweave

Pass 1: for each original node x , create a copy x' and insert it between x and $x.\text{next}$, forming $x \rightarrow x' \rightarrow x.\text{next} \rightarrow \dots$ **Pass 2:** set each copy’s `random` via $x'.\text{random} = x.\text{random}.\text{next}$ – because every original is immediately followed by its copy, the copy of any target is one `next` away. **Pass 3:** separate the interleaved list back into the original and the cloned list, restoring `next` pointers.

The Copy Sits Next to Its Original

Interleaving means “the clone of node x ” is always $x \rightarrow \text{next}$. So the `random` pointer of a clone is $x \rightarrow \text{random} \rightarrow \text{next}$ – computable without a hash map, which is what removes the $O(n)$ auxiliary space a map-based clone would use.

Algorithm

1. Insert each node’s copy immediately after it.
2. For each copy, set `copy->random = orig->random->next` (guard null).
3. Detach the copies into a separate list, restoring the originals’ `next`.

Dry run

Dry Run. $A \rightarrow B$ with $A.\text{random} = B$, $B.\text{random} = B$.

- Interleave: $A \rightarrow A' \rightarrow B \rightarrow B'$. Set $A'.\text{random} = A.\text{random}.\text{next} = B'$; $B'.\text{random} = B'$.
- Unweave: clone is $A' \rightarrow B'$ with correct randoms.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Node {
5      int val;
6      Node* next;
7      Node* random;
8      Node(int v) : val(v), next(nullptr), random(nullptr) {}
9  };
10
11 Node* copyRandomList(Node* head) {
12     if (!head) return nullptr;
13     // Pass 1: interleave copies
14     for (Node* c = head; c; c = c->next->next) {
15         Node* copy = new Node(c->val);
16         copy->next = c->next;
17         c->next = copy;
18     }
19     // Pass 2: wire random pointers
20     for (Node* c = head; c; c = c->next->next)
21         if (c->random) c->next->random = c->random->next;
22     // Pass 3: unweave
23     Node dummy(0), *t = &dummy;
24     for (Node* c = head; c; c = c->next) {
25         t->next = c->next;           // pick the copy
26         t = t->next;
27         c->next = c->next->next;     // restore original
28     }
29     return dummy.next;
30 }
31
32 int main() {
33     Node* a = new Node(1);
34     Node* b = new Node(2);
35     a->next = b;
36     a->random = b; b->random = b;
37     Node* cl = copyRandomList(a);
38     for (Node* c = cl; c; c = c->next)
39         cout << c->val << "(rand " << (c->random ? c->random->val : -1)
40             << ") ";
41     cout << "\n"; // 1(rand 2) 2(rand
42 }

```

Complexity Analysis

Time Complexity. $O(n)$ – three linear passes. **Space Complexity.** $O(1)$ extra (versus $O(n)$ for a hash-map clone).

Key Takeaway

Cloning with random pointers in $O(1)$ space uses interleaving so each clone sits beside its original, making random wiring a local `->next` lookup. It is the standout example of exploiting the list's own structure instead of an auxiliary map.

Chapter 2

Strings

A string is an array of characters, so array techniques – two pointers, frequency counts, prefix sums – carry straight over. What is special is the family of pattern-matching algorithms (KMP, Z, Rabin–Karp) built on one idea: a string’s own borders (prefixes that are also suffixes) let you skip re-comparison. This chapter runs from parsing puzzles to those linear-time matchers.

2.1 From Character Counting to Border Theory

Most easy string problems are careful scanning or counting; the harder ones are exact pattern matching, where naive comparison is $O(nm)$ and the prefix/Z functions bring it to $O(n + m)$.

String Technique Map

- **Parsing / building strings** – a single scan with a counter or stack: Remove Outermost Parentheses, Maximum Nesting Depth, Roman to Integer, Atoi, Count and Say.
- **Two pointers / reversal** – Reverse Words, Largest Odd Number, Longest Common Prefix.
- **Frequency counting / hashing** – Anagram Check, Isomorphic Strings, Sort by Frequency, Sum of Beauty.
- **Expand around centre** – Longest Palindromic Substring in $O(n^2)$ without DP.
- **Border-based matching (prefix/Z function)** – Rabin–Karp, Z-Function, KMP/LPS, Shortest Palindrome, Longest Happy Prefix.

The One Idea Behind Fast Matching

A *border* is a proper prefix that is also a suffix. The prefix function (KMP) and Z-function both compute, in linear time, how much of the pattern reappears – so after a mismatch you resume from the longest border instead of restarting. Every hard problem here is an application of that.

2.2 Remove Outermost Parentheses

Problem Statement. A valid parentheses string decomposes into primitive groups. Remove the outermost pair of parentheses of every primitive group and return the result. For example $((()())())$ becomes $()()()$.

Pattern Recognition

Tracking parenthesis nesting is a **depth counter**: an open bracket raises the depth, a close lowers it. A bracket is “outermost” exactly when it sits at depth transitioning to/from 0 – keep every other bracket.

Keep brackets that are not at the outer layer

Scan left to right maintaining a depth counter. For an open bracket, it is outermost iff the current depth is 0 (before incrementing); increment after. For a close bracket, decrement first, and it is outermost iff the depth returns to 0. Append every bracket *except* those outermost ones. This peels exactly one layer off each primitive group.

Depth Zero Marks the Outer Layer

A primitive group starts when depth goes $0 \rightarrow 1$ and ends when it returns $1 \rightarrow 0$; those two brackets are its outermost pair. Everything strictly inside has depth ≥ 1 , so a simple counter distinguishes what to keep.

Algorithm

1. `depth = 0`, empty result.
2. For each char: if `'('`, append it only when `depth > 0`, then increment; if `')'`, decrement, then append only when `depth > 0`.
3. Return the result.

Dry run

Dry Run. $((()())())$.

- First group $((()()))$: drop the outer pair, keep $()()$. Second $((()))$: keep $()$.
- Result $()()()$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string removeOuterParentheses(const string& s) {
5     string res;
6     int depth = 0;
7     for (char c : s) {
8         if (c == '(') {
9             if (depth > 0) res += c;           // not outermost
10            depth++;
11        } else {
12            depth--;

```

```

13         if (depth > 0) res += c;
14     }
15 }
16 return res;
17 }
18
19 int main() {
20     cout << removeOuterParentheses("(()())(())") << "\n";    // ()()()
21     return 0;
22 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one scan. **Space Complexity.** $O(n)$ for the output.

Key Takeaway

Parenthesis-structure problems are usually a depth counter, not a stack: depth 0 marks group boundaries. This counter idea reappears in maximum nesting depth and bracket-validity checks.

2.3 Reverse Words in a String

Problem Statement. Given a string of words separated by spaces (possibly with leading, trailing, or multiple spaces), return the words in reverse order, separated by single spaces.

Pattern Recognition

“Reverse the order of words” is a parse-and-rebuild: extract words by scanning, then emit them back-to-front. Handling stray spaces is the only subtlety, cleanly solved with a string stream or a manual scan.

Split, then join in reverse

Scan the string, collecting maximal non-space runs as words while skipping any spaces. Store the words, then output them from last to first joined by a single space. This normalises whitespace automatically – extra and edge spaces vanish because only actual words are collected.

Collect Words, Ignore the Spaces

Treating the string as a sequence of words rather than characters sidesteps messy space handling: gather each word once, discard the gaps, and reorder. The single-space join produces clean output regardless of the input’s spacing.

Algorithm

1. Read words one at a time (a stream skips runs of spaces).
2. Push each word; then emit them in reverse with single-space separators.

Dry run

Dry Run. " the sky is blue "

- Words: the, sky, is, blue.

- Reversed and joined: "blue is sky the".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string reverseWords(const string& s) {
5     stringstream ss(s);
6     string word;
7     vector<string> words;
8     while (ss >> word) words.push_back(word);           // skips extra
        spaces
9     string res;
10    for (int i = (int)words.size() - 1; i >= 0; i--) {
11        res += words[i];
12        if (i) res += ' ';
13    }
14    return res;
15 }
16
17 int main() {
18     cout << "[" << reverseWords("  the sky is  blue ") << "]\n"; // [
        blue is sky the]
19     return 0;
20 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ for the words and output.

Key Takeaway

Word-level problems become easy when you tokenise first and treat words as units; a stringstream handles arbitrary whitespace for free. In-place $O(1)$ -space variants reverse the whole string then each word.

2.4 Largest Odd Number in a String

Problem Statement. Given a string of digits, return the largest odd number that is a non-empty prefix substring (a prefix of the string), or the empty string if none exists.

Pattern Recognition

A number is odd iff its *last* digit is odd. Since we may only take a prefix, the largest odd prefix ends at the **rightmost odd digit** – so scan from the right for the first odd digit and return everything up to it.

Find the rightmost odd digit

Any prefix of the string is a candidate number. To be odd it must end in an odd digit, and to be largest it should be as long as possible. So scan from the right end leftward for the first odd digit; the prefix ending there is the answer. If no digit is odd, no odd prefix exists and the answer is empty.

Oddness Depends Only on the Last Digit

Because parity is decided by the final digit, the longest odd prefix is the one ending at the rightmost odd digit. A single right-to-left scan finds it; leading digits never change oddness, so length is maximised automatically.

Algorithm

1. Scan from the last index leftward.
2. At the first odd digit (index i), return the substring $[0, i]$.
3. If none found, return the empty string.

Dry run

Dry Run. "35427" and "5468".

- "35427": last digit 7 is odd $\rightarrow$ whole string "35427".
- "5468": rightmost odd is 5 at index 0 $\rightarrow$ "5".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string largestOddNumber(const string& s) {
5     for (int i = (int)s.size() - 1; i >= 0; i--)
6         if ((s[i] - '0') % 2 == 1) return s.substr(0, i + 1);    // odd
7     return "";    // prefix
8 }
9
10 int main() {
11     cout << "[" << largestOddNumber("35427") << "]\n";    // [35427]
12     cout << "[" << largestOddNumber("5468") << "]\n";    // [5]
13     return 0;
14 }
```

Complexity Analysis

Time Complexity. $O(n)$ – one scan. **Space Complexity.** $O(1)$ beyond the returned substring.

Key Takeaway

When a property depends only on a boundary character (here, oddness on the last digit), scan toward that boundary rather than evaluating every prefix. The rightmost-odd-digit observation collapses the problem to one pass.

2.5 Longest Common Prefix

Problem Statement. Given an array of strings, find the longest common prefix shared by all of them. Return the empty string if there is none.

Pattern Recognition

“Common prefix of all strings” is a **vertical scan**: compare the characters at each position across all strings simultaneously; the first column where they disagree (or where any string ends) marks the end of the common prefix.

Compare column by column

Take the first string as a reference. For each character position j , check that every other string has the same character at j and is long enough. The moment a mismatch or a too-short string appears, the common prefix is everything before position j . This stops as early as possible, touching only the characters within the eventual prefix plus one mismatch column.

The Shortest String Bounds the Prefix

The common prefix cannot exceed the shortest string, and any single mismatch in a column ends it. A vertical (position-by-position) scan finds that stopping point directly, without sorting or full pairwise comparison.

Algorithm

1. For each position j of the first string: let c be its character.
2. If any other string is shorter than $j + 1$ or differs at j , return the prefix $[0, j)$.
3. If the loop completes, the whole first string is the prefix.

Dry run

Dry Run. ["flower", "flow", "flight"].

- Column 0: all f. Column 1: all l. Column 2: o,o,i – mismatch.
- Common prefix "fl".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string longestCommonPrefix(const vector<string>& strs) {
5     if (strs.empty()) return "";
6     for (int j = 0; j < (int)strs[0].size(); j++) {
7         char c = strs[0][j];
8         for (const string& s : strs)
9             if (j >= (int)s.size() || s[j] != c)
10                 return strs[0].substr(0, j);    // first
11                                                     disagreement
12     }
13     return strs[0];
14 }
15
16 int main() {
17     cout << "[" << longestCommonPrefix({"flower", "flow", "flight"}) <<
18         "]"<< "\n";    // [fl]
19     return 0;
20 }
```

18 }

Complexity Analysis

Time Complexity. $O(\sum |s_i|)$ worst case; typically far less, stopping at the first mismatch.
Space Complexity. $O(1)$ beyond the result.

Key Takeaway

The longest common prefix is a vertical scan that halts at the first mismatching column. Comparing position-by-position across all strings is simpler and often faster than sorting or divide-and-conquer alternatives.

2.6 Isomorphic Strings

Problem Statement. Two strings are isomorphic if the characters of one can be mapped to the characters of the other so that the mapping is a consistent one-to-one correspondence preserving order. Determine whether two strings are isomorphic.

Pattern Recognition

Isomorphism is a **bijective character mapping**: each character of s must map to a single character of t *and* vice versa. Two mapping tables (forward and backward) enforce the one-to-one requirement in one pass.

Two consistent maps

Walk both strings together. For each position, check the forward map $s[i] \rightarrow t[i]$: if $s[i]$ was seen before, its recorded image must equal $t[i]$; likewise the backward map $t[i] \rightarrow s[i]$ must be consistent. Requiring *both* directions prevents two different source characters from mapping to the same target (e.g. “ab”→“aa” is rejected). Any inconsistency means not isomorphic.

One-to-One Needs Both Directions Checked

A single forward map allows collisions (many-to-one). Enforcing the backward map too makes the correspondence bijective, which is what “isomorphic” demands. Both maps updated in lockstep catch every violation.

Algorithm

1. If lengths differ, return false.
2. For each i : verify and record $s[i] \rightarrow t[i]$ and $t[i] \rightarrow s[i]$; on any conflict return false.
3. Return true.

Dry run

Dry Run. “egg” and “add”.

- $e \rightarrow a, g \rightarrow d, g \rightarrow d$ consistent; backward $a \rightarrow e, d \rightarrow g$ consistent.
- Isomorphic. (“foo”→“bar” fails: $o \rightarrow a$ then $o \rightarrow r$.)

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isIsomorphic(const string& s, const string& t) {
5     if (s.size() != t.size()) return false;
6     unordered_map<char, char> fwd, bwd;
7     for (int i = 0; i < (int)s.size(); i++) {
8         char a = s[i], b = t[i];
9         if (fwd.count(a) && fwd[a] != b) return false;
10        if (bwd.count(b) && bwd[b] != a) return false;
11        fwd[a] = b;
12        bwd[b] = a;
13    }
14    return true;
15 }
16
17 int main() {
18     cout << isIsomorphic("egg", "add") << "\n";    // 1
19     cout << isIsomorphic("foo", "bar") << "\n";    // 0
20     return 0;
21 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$ (at most 256 character keys).

Key Takeaway

Isomorphism is a bijection check, requiring *both* forward and backward consistency to rule out many-to-one collisions. Whenever a problem says “one-to-one correspondence,” remember to guard both directions.

2.7 Check if One String is a Rotation of Another

Problem Statement. Given two strings s and $goal$, return true if s can become $goal$ after some number of left (or right) rotations.

Pattern Recognition

The one-line trick: $goal$ is a rotation of s iff they have the same length and $goal$ is a **substring of $s + s$** . Doubling s contains every rotation of s as a window.

Every rotation lives inside $s + s$

Concatenating s with itself produces a string in which every cyclic rotation of s appears as a contiguous length- $|s|$ substring. So if $goal$ has the same length as s and occurs somewhere in $s + s$, it is a rotation; otherwise it is not. A single substring search decides it.

Doubling Exposes All Rotations

Rotating s moves a prefix to the back; $s + s$ contains all such prefix-suffix recombinations as windows. Checking membership in $s + s$ therefore tests all rotations at once, avoiding generating each one.

Algorithm

1. If $|s| \neq |goal|$, return false.
2. Return whether $goal$ is a substring of $s + s$.

Dry run

Dry Run. $s = \text{"abcde"}$, $goal = \text{"cdeab"}$.

- $s + s = \text{"abcdeabcde"}$ contains "cdeab" starting at index 2.
- Same length and found $\Rightarrow$ rotation.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool rotateString(const string& s, const string& goal) {
5     if (s.size() != goal.size()) return false;
6     return (s + s).find(goal) != string::npos;    // goal is a window
7     of s+s
8 }
9
10 int main() {
11     cout << rotateString("abcde", "cdeab") << "\n";    // 1
12     cout << rotateString("abcde", "abced") << "\n";    // 0
13     return 0;
14 }
```

Complexity Analysis

Time Complexity. $O(n^2)$ with naive find, or $O(n)$ using a linear substring search (KMP).
Space Complexity. $O(n)$ for $s + s$.

Key Takeaway

Rotation testing reduces to substring search in the doubled string – a recurring “concatenate to expose cyclic structure” idea also seen in minimal-rotation and circular-array problems. Pair it with KMP for linear time.

2.8 Check if Two Strings are Anagrams

Problem Statement. Two strings are anagrams if one is a rearrangement of the other – the same characters with the same frequencies. Determine whether two strings are anagrams.

Pattern Recognition

Anagram checking is **frequency counting**: two strings are anagrams iff every character occurs the same number of times in both. A single count array (increment for one string, decrement for the other) that ends all zeros confirms it.

One count array, up then down

If the lengths differ, they cannot be anagrams. Otherwise keep a count array over the alphabet: increment for each character of the first string and decrement for each character of the second. If the strings are anagrams, every increment is matched by a decrement, so all counts return to zero. Any nonzero entry means a character mismatch in frequency.

Anagram iff All Frequency Differences Are Zero

Rearrangement preserves multiset membership, so the per-character counts must match exactly. Folding both strings into one signed count array reduces the test to “are all counts zero,” in one pass and constant space.

Algorithm

1. If lengths differ, return false.
2. Increment counts for the first string, decrement for the second.
3. Return true iff every count is zero.

Dry run

Dry Run. "anagram" and "nagaram".

- Both have three a, one n, one g, one r, one m.
- All counts cancel to zero – anagrams.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isAnagram(const string& s, const string& t) {
5     if (s.size() != t.size()) return false;
6     int cnt[26] = {0};
7     for (char c : s) cnt[c - 'a']++;
8     for (char c : t) cnt[c - 'a']--;
9     for (int x : cnt) if (x != 0) return false;
10    return true;
11 }
12
13 int main() {
14     cout << isAnagram("anagram", "nagaram") << "\n";    // 1
15     cout << isAnagram("rat", "car") << "\n";            // 0
16     return 0;
17 }
```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$ – a fixed alphabet-sized array (sorting both strings is an $O(n \log n)$ alternative).

Key Takeaway

Anagram detection is a signed frequency count that must zero out. The increment-then-decrement pattern is the cleanest way to compare two multisets of characters in linear time and constant space.

2.9 Sort Characters by Frequency

Problem Statement. Given a string, rearrange its characters so that they appear in decreasing order of frequency. Characters with the same frequency may be in any order; each character is repeated as many times as it occurs.

Pattern Recognition

“Order by how often each appears” is **count then sort**: tally each character’s frequency, sort the distinct characters by descending count, and rebuild the string by repeating each character its frequency times.

Tally, sort, rebuild

First count occurrences of each character with a hash map or array. Collect the (character, count) pairs and sort them by count in descending order. Finally, construct the result by appending each character **count** times in that order. The heavy-hitters come first, exactly as required.

Frequency Is the Sort Key

The problem is a sort where the key is a character’s frequency, not its value. Computing all frequencies up front turns it into a standard comparison sort over a small set of distinct characters.

Algorithm

1. Count each character’s frequency.
2. Sort distinct characters by descending frequency.
3. Append each character **frequency** times to the output.

Dry run

Dry Run. "tree".

- Counts: $e \times 2, t \times 1, r \times 1$.
- Sorted by count: e first $\rightarrow$ "eetr" (or "eert").

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string frequencySort(const string& s) {
5     unordered_map<char, int> freq;

```

```

6   for (char c : s) freq[c]++;
7   vector<pair<int, char>> v;           // {count, char}
8   for (auto& [c, f] : freq) v.push_back({f, c});
9   sort(v.rbegin(), v.rend());         // descending by
    count
10
11   string res;
12   for (auto& [f, c] : v) res.append(f, c);
13   return res;
14 }
15
16 int main() {
17     cout << frequencySort("tree") << "\n"; // eetr (e's first)
18     return 0;
19 }

```

Complexity Analysis

Time Complexity. $O(n + k \log k)$ where k is the number of distinct characters. **Space Complexity.** $O(n)$ for counts and output.

Key Takeaway

Frequency ordering is count-then-sort with frequency as the key. A bucket-by-count approach avoids the sort for $O(n)$, but the tally-and-sort form is the simplest and generalises to top- k frequent element problems.

2.10 Maximum Nesting Depth of Parentheses

Problem Statement. Given a valid parentheses expression (possibly with digits and operators), return the maximum nesting depth of the parentheses.

Pattern Recognition

Nesting depth is a **running counter**: increment on ' $($ ', decrement on ' $)$ ', and track the maximum value the counter reaches. Non-bracket characters are ignored.

Track the counter's peak

Scan the string once. Each open bracket deepens the nesting (increment the counter); each close bracket exits one level (decrement). The answer is the highest value the counter attains during the scan. Because the expression is valid, the counter never goes negative and ends at zero.

Depth Is the Counter, Answer Is Its Maximum

The current nesting level is exactly the number of unmatched open brackets so far – the counter. The deepest nesting is that counter's peak, captured by a single `max` update per open bracket.

Algorithm

1. `depth = 0, best = 0.`
2. For each char: ' $($ ' increments `depth` and updates `best`; ' $)$ ' decrements `depth`.

3. Return best.

Dry run

Dry Run. "(1+(2*3)+((8)/4))+1".

- Depth rises to 3 at the innermost ((8)) region.
- Maximum depth = 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int maxDepth(const string& s) {
5     int depth = 0, best = 0;
6     for (char c : s) {
7         if (c == '(') best = max(best, ++depth);
8         else if (c == ')') depth--;
9     }
10    return best;
11 }
12
13 int main() {
14     cout << maxDepth("(1+(2*3)+((8)/4))+1") << "\n";    // 3
15     return 0;
16 }
```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

Maximum nesting depth is the peak of a bracket counter – no stack needed for a guaranteed-valid expression. This counter idea is the same one behind removing outermost parentheses; both track unmatched-open count.

2.11 Roman Numeral to Integer

Problem Statement. Convert a Roman numeral string (using symbols I, V, X, L, C, D, M) to its integer value.

Pattern Recognition

Roman numerals are almost additive, with a **subtractive exception**: a smaller symbol before a larger one is subtracted (IV = 4). So scan left to right and, at each symbol, *subtract* its value if the next symbol is larger, otherwise add it.

Add, unless the next is larger

Map each symbol to its value. Walk the string; for each symbol, compare its value to the next symbol's value. If the current value is smaller than the next, it is part of a subtractive pair (like

IX), so subtract it; otherwise add it. The last symbol is always added. This one pass handles both additive and subtractive forms without special-casing each pair.

A Smaller Symbol Before a Larger One Subtracts

The only irregularity in Roman numerals is the subtractive pair. Detecting it locally – “current value < next value” – converts the whole numeral in a single left-to-right pass.

Algorithm

1. Map symbols to values.
2. For each position i : if $\text{val}[i] < \text{val}[i+1]$, subtract $\text{val}[i]$, else add it.
3. Return the accumulated total.

Dry run

Dry Run. "MCMXCIV" (1994).

- M+1000; C<M so -100; M+1000; X<C so -10; C+100; I<V so -1; V+5.
- Total = $1000 - 100 + 1000 - 10 + 100 - 1 + 5 = 1994$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int romanToInt(const string& s) {
5     unordered_map<char, int> val = {
6         {'I', 1}, {'V', 5}, {'X', 10}, {'L', 50}, {'C', 100}, {'D', 500}, {'M',
7             , 1000}
8     };
9     int total = 0, n = s.size();
10    for (int i = 0; i < n; i++) {
11        if (i + 1 < n && val[s[i]] < val[s[i + 1]]) total -= val[s[i]];
12        else total += val[s[i]];
13    }
14    return total;
15 }
16
17 int main() {
18     cout << romanToInt("MCMXCIV") << "\n";           // 1994
19     cout << romanToInt("LVIII") << "\n";              // 58
20     return 0;
21 }
```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$ (a fixed symbol table).

Key Takeaway

Roman-to-integer is additive with one subtractive rule, detected by comparing each symbol to its successor. Handling the exception locally avoids enumerating every subtractive pair – a clean parsing pattern.

2.12 Implement Atoi

Problem Statement. Implement `atoi`: convert a string to a 32-bit signed integer. Skip leading whitespace, read an optional sign, then read digits until a non-digit; clamp the result to the 32-bit signed range on overflow.

Pattern Recognition

This is a careful **state-by-state parse**: whitespace, then optional sign, then digits. The one algorithmic subtlety is detecting overflow *before* it happens, by comparing against the limit divided by ten.

Parse in stages, guard overflow

Advance past leading spaces. Read an optional '+' or '-' to fix the sign. Then accumulate digits: for each digit d , the new value would be $10 \cdot \text{value} + d$. Before doing this, check whether it would exceed the 32-bit limit; if so, return the clamped extreme (`INT_MAX` or `INT_MIN`). Stop at the first non-digit.

Check Overflow Against `LIMIT/10` First

Rather than overflowing and then detecting it, test whether $\text{value} > \text{INT_MAX}/10$ (or equal with a large next digit) *before* multiplying. This pre-check keeps the arithmetic within range and clamps correctly at the boundary.

Algorithm

1. Skip spaces; read optional sign.
2. For each digit: if appending it would overflow, return the clamped limit; else update `value`.
3. Return `sign * value`.

Dry run

Dry Run. " -42abc" and "2147483648".

- " -42abc": skip spaces, sign -, digits 42, stop at a $\rightarrow -42$.
- "2147483648" exceeds `INT_MAX` $\rightarrow$ clamp to 2147483647.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int myAtoi(const string& s) {
5     int i = 0, n = s.size();
6     while (i < n && s[i] == ' ') i++;           // skip whitespace
7     int sign = 1;
8     if (i < n && (s[i] == '+' || s[i] == '-')) sign = (s[i++] == '-') ?
        -1 : 1;
9
10    long long value = 0;
11    while (i < n && isdigit(s[i])) {
12        value = value * 10 + (s[i] - '0');

```

```

13     if (sign == 1 && value > INT_MAX) return INT_MAX;    // clamp
14     if (sign == -1 && -value < INT_MIN) return INT_MIN;
15     i++;
16 }
17 return (int)(sign * value);
18 }
19
20 int main() {
21     cout << myAtoi("  -42abc") << "\n";                // -42
22     cout << myAtoi("2147483648") << "\n";                // 2147483647
23     return 0;
24 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

Atoi is a staged parse – spaces, sign, digits – with an overflow guard checked before each multiply. Using a wider accumulator (`long long`) and clamping at the boundary is the robust way to handle 32-bit overflow.

2.13 Longest Palindromic Substring (Expand Around Centre)

Problem Statement. Given a string, find its longest substring that is a palindrome, without dynamic programming.

Pattern Recognition

“Longest palindromic substring” is solved by **expand around centre**: every palindrome has a centre, so try each of the $2n - 1$ possible centres (each character and each gap) and expand outward while the characters match, tracking the longest.

Grow palindromes from each centre

A palindrome is symmetric about its centre, which is either a single character (odd length) or the gap between two characters (even length). For each of the $2n - 1$ centres, expand two pointers outward while they stay in bounds and the characters are equal; the span they reach is the longest palindrome centred there. Keep the overall longest across all centres. This $O(n^2)$ method needs no DP table.

Try Every Centre, Odd and Even

Because a palindrome is determined by its centre and radius, enumerating all centres and expanding greedily finds the maximum. Remember the even case (centre between two characters), which the odd-only expansion would miss.

Algorithm

1. For each index i : expand around centre (i, i) (odd) and $(i, i + 1)$ (even).
2. Each expansion returns the palindrome length; keep the longest and its start.
3. Return the best substring.

Dry run

Dry Run. "babad".

- Centre at a (index 1) expands to "bab"; centre at a (index 3) expands to "aba".
- Longest length 3 ("bab" or "aba").

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string longestPalindrome(const string& s) {
5     if (s.empty()) return "";
6     int start = 0, maxlen = 1;
7     auto expand = [&](int l, int r) {
8         while (l >= 0 && r < (int)s.size() && s[l] == s[r]) { l--; r++; }
9         if (r - l - 1 > maxlen) { maxlen = r - l - 1; start = l + 1; }
10    };
11    for (int i = 0; i < (int)s.size(); i++) {
12        expand(i, i);           // odd length
13        expand(i, i + 1);       // even length
14    }
15    return s.substr(start, maxlen);
16 }
17
18 int main() {
19     cout << longestPalindrome("babad") << "\n";    // bab (or aba)
20     cout << longestPalindrome("cbbd") << "\n";     // bb
21     return 0;
22 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ – $2n - 1$ centres, each expanding up to $O(n)$. **Space Complexity.** $O(1)$ (Manacher's algorithm achieves $O(n)$).

Key Takeaway

Expand-around-centre solves longest palindromic substring in $O(n^2)$ with no DP table, handling odd and even centres. When linear time is required, Manacher's algorithm computes all palindromic radii in $O(n)$.

2.14 Sum of Beauty of All Substrings

Problem Statement. The beauty of a string is the difference between the highest and lowest character frequencies in it (considering only characters that appear). Compute the sum of the beauty of all substrings.

Pattern Recognition

There are $O(n^2)$ substrings, so an $O(n^2)$ approach is expected: fix each start, then **extend the end one character at a time, maintaining a frequency array**. After each

extension, the beauty (max minus min nonzero frequency) is computed and added.

Grow substrings, update frequencies incrementally

For each starting index i , reset a frequency array and extend the end j from i to the end. Adding $s[j]$ updates one frequency in $O(1)$. The beauty of substring $[i, j]$ is the maximum frequency minus the minimum *nonzero* frequency. Scanning the 26 counts to find those extremes is $O(26)$ per substring, giving $O(26n^2)$ overall – fast enough for the constraints.

Reuse the Frequency Array as the Window Grows

Fixing the start and extending the end lets each new substring's frequencies be obtained by a single increment, avoiding a recount. Only the max/min scan over the fixed 26-letter alphabet is repeated, keeping the inner cost constant.

Algorithm

1. For each start i : clear a 26-count array.
2. For each end $j \geq i$: increment `count[s[j]]`; compute `beauty = max - minnonzero` over the counts; add to the total.
3. Output the total.

Dry run

Dry Run. "aabcb".

- Most substrings have beauty 0 (uniform or single char); "aabcb" and "aab" contribute positive beauty.
- Summing all substrings' beauties gives 5.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int beautySum(const string& s) {
5     int n = s.size(), total = 0;
6     for (int i = 0; i < n; i++) {
7         int cnt[26] = {0};
8         for (int j = i; j < n; j++) {
9             cnt[s[j] - 'a']++;
10            int mx = 0, mn = INT_MAX;
11            for (int k = 0; k < 26; k++) if (cnt[k]) {
12                mx = max(mx, cnt[k]);
13                mn = min(mn, cnt[k]);
14            }
15            total += mx - mn;
16        }
17    }
18    return total;
19 }
20

```

```

21 int main() {
22     cout << beautySum("aabcb") << "\n";           // 5
23     return 0;
24 }

```

Complexity Analysis

Time Complexity. $O(26n^2)$ – all substrings, each an $O(26)$ extreme scan. **Space Complexity.** $O(1)$ (a fixed count array).

Key Takeaway

When all $O(n^2)$ substrings must be examined, fix the start and extend the end while maintaining an incremental frequency array – only the small fixed-alphabet extreme scan repeats. This “grow the window, update counts” pattern recurs in substring-statistic problems.

2.15 Count and Say

Problem Statement. The count-and-say sequence starts with "1". Each term is generated by reading the previous term as runs of identical digits, saying each run as its count followed by the digit. Given n , output the n -th term. For example the terms are 1, 11, 21, 1211, 111221.

Pattern Recognition

Each term is a **run-length encoding** of the previous one: scan the previous string, group consecutive equal digits, and emit “count digit” for each group. Repeat $n - 1$ times starting from "1".

Run-length encode, repeatedly

Start with "1". To produce the next term, walk the current string counting the length of each maximal run of identical characters; append the run length followed by the character. Applying this $n - 1$ times yields the n -th term. Each step reads and describes the previous one – the sequence is literally “say what you see.”

Say the Runs You See

The transformation from one term to the next is exactly run-length encoding: consecutive duplicates collapse into a count-plus-digit pair. Iterating this description operation generates the sequence.

Algorithm

1. `cur = "1"`.
2. Repeat $n - 1$ times: scan `cur`, and for each run of length c of digit d append c then d to a new string; set `cur` to it.
3. Return `cur`.

Dry run

Dry Run. $n = 4$.

- "1" → one 1 → "11" → two 1s → "21" → one 2 one 1 → "1211".
- The 4th term is "1211".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string countAndSay(int n) {
5     string cur = "1";
6     for (int t = 1; t < n; t++) {
7         string next;
8         int i = 0, m = cur.size();
9         while (i < m) {
10             int j = i;
11             while (j < m && cur[j] == cur[i]) j++;    // run of equal
12             next += to_string(j - i);                // count
13             next += cur[i];                          // digit
14             i = j;
15         }
16         cur = next;
17     }
18     return cur;
19 }
20
21 int main() {
22     cout << countAndSay(4) << "\n";                // 1211
23     cout << countAndSay(5) << "\n";                // 111221
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. $O(n \cdot L)$ where L is the length of the longest term (which grows roughly geometrically). **Space Complexity.** $O(L)$.

Key Takeaway

Count-and-say is iterated run-length encoding – each term describes the runs of the previous one. Recognising the “say what you see” rule as RLE turns a puzzling sequence into a simple grouping scan.

2.16 Minimum Bracket Reversals to Balance

Problem Statement. Given a string of only '{' and '}', find the minimum number of bracket reversals needed to make it balanced, or -1 if it is impossible.

Pattern Recognition

Cancel matched pairs first; what remains is always of the form $\}} \dots \{ \{ \dots$ – some unmatched closes followed by some unmatched opens. The answer is a simple formula over those two counts. An odd-length string can never balance.

Reduce, then count the leftovers

Scan the string like bracket matching: a $\}$ that has a preceding unmatched $\{$ cancels it; otherwise it is an unmatched close. After the scan you are left with c unmatched closes and o unmatched opens. To fix them, each pair of same-direction unmatched brackets needs one reversal, plus a possible extra when a count is odd: $\lceil c/2 \rceil + \lceil o/2 \rceil$. If the total length is odd, balancing is impossible (-1).

The Residue Is Always $\}} \dots \{ \{ \dots$

Matching cancels every properly nested pair, leaving unmatched closes on the left and unmatched opens on the right. Reversing brackets in pairs fixes them, with a ceiling to handle a single leftover of each kind.

Algorithm

1. If the length is odd, return -1 .
2. Scan, cancelling each $\}$ against a pending $\{$; count unmatched opens o and closes c .
3. Return $\lceil c/2 \rceil + \lceil o/2 \rceil$.

Dry run

Dry Run. $\}\{\{\}\}\{\{\{$.

- After cancelling matched pairs, $c = 1$ unmatched close, $o = 3$ unmatched opens.
- Answer $\lceil 1/2 \rceil + \lceil 3/2 \rceil = 1 + 2 = 3$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minReversals(const string& s) {
5     if (s.size() % 2 != 0) return -1;
6     int open = 0, close = 0; // unmatched counts
7     for (char c : s) {
8         if (c == '{') open++;
9         else { // '}'
10             if (open > 0) open--; // cancel a pending '{'
11             else close++; // unmatched close
12         }
13     }
14     return (open + 1) / 2 + (close + 1) / 2; // ceilings
15 }
16

```

```

17 int main() {
18     cout << minReversals("}{}}}{{{") << "\n";           // 3
19     cout << minReversals("{{{{") << "\n";               // 2
20     return 0;
21 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(1)$.

Key Takeaway

Bracket balancing reduces to counting unmatched opens and closes after cancellation, then a ceiling formula. The insight that the residue is always $\}\dots\{\dots$ turns the problem into simple arithmetic.

2.17 Rabin–Karp Pattern Matching

Problem Statement. Given a text and a pattern, find all positions where the pattern occurs in the text, using hashing.

Pattern Recognition

Rabin–Karp compares a **rolling hash** of each text window against the pattern's hash. Sliding the window updates the hash in $O(1)$ (remove the leaving character, add the entering one), so all windows are hashed in linear time; only hash matches are verified character by character.

Rolling hash over windows

Treat a length- m string as a base- B number modulo a large prime. Compute the pattern's hash and the first window's hash. To slide the window one step, subtract the contribution of the outgoing character (times B^{m-1}), multiply by B , and add the incoming character – an $O(1)$ update. When a window's hash equals the pattern's, confirm with a direct comparison to rule out the rare hash collision.

Slide the Hash Instead of Recomputing It

Because a window hash differs from the next only in its first and last characters, it updates in constant time. This turns naive $O(nm)$ matching into $O(n + m)$ expected time, with verification guarding against collisions.

Algorithm

1. Compute the pattern hash and the first text-window hash; precompute B^{m-1} .
2. Slide across the text, updating the hash in $O(1)$; on a hash match, verify the substring.
3. Report all confirmed positions.

Dry run

Dry Run. text "abcabc", pattern "bc".

- Window hashes for ab, bc, ca, ab, bc; matches at indices 1 and 4.

- Occurrences: positions 1 and 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef unsigned long long ull;
4 const ull B = 257, MOD = 1000000007ULL;
5
6 vector<int> rabinKarp(const string& text, const string& pat) {
7     int n = text.size(), m = pat.size();
8     vector<int> res;
9     if (m > n) return res;
10
11     ull patHash = 0, winHash = 0, power = 1;
12     for (int i = 0; i < m; i++) {
13         patHash = (patHash * B + pat[i]) % MOD;
14         winHash = (winHash * B + text[i]) % MOD;
15         if (i) power = (power * B) % MOD;           //  $B^{(m-1)}$ 
16     }
17
18     for (int i = 0; i + m <= n; i++) {
19         if (winHash == patHash && text.compare(i, m, pat) == 0)
20             res.push_back(i);                       // verify to avoid
21                                                     // collision
22         if (i + m < n) {                             // roll the window
23             winHash = (winHash + MOD - text[i] * power % MOD) % MOD;
24             winHash = (winHash * B + text[i + m]) % MOD;
25         }
26     }
27     return res;
28
29 int main() {
30     for (int p : rabinKarp("abcabc", "bc")) cout << p << " ";
31     cout << "\n";                                   // 1 4
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ expected; $O(nm)$ worst case if many hash collisions force verification. **Space Complexity.** $O(1)$.

Key Takeaway

Rabin–Karp slides a rolling hash to compare windows in constant time, verifying only on hash equality. Choosing a good base and prime keeps collisions rare; it shines for multi-pattern search and 2D matching.

2.18 Z-Function and Pattern Matching

Problem Statement. Compute the Z-function of a string – for each index, the length of the longest substring starting there that matches a prefix of the whole string – and use it to find all occurrences of a pattern in a text.

Pattern Recognition

The **Z-function** measures self-similarity in linear time using a maintained “Z-box” (the rightmost prefix-match interval). For pattern matching, run it on $pattern + \# + text$: positions in the text part with $Z = |pattern|$ are matches.

The Z-box makes it linear

$Z[i]$ is how far the substring at i agrees with the prefix. Maintain the interval $[l, r]$ of the rightmost prefix match found so far. If i lies inside $[l, r]$, the mirror value $Z[i - l]$ gives a free lower bound for $Z[i]$ (capped by $r - i$); then extend by direct comparison, moving r forward. Each character is compared a constant number of times amortised, so the whole array is $O(n)$.

Reuse the Rightmost Match to Skip Comparisons

Inside the current Z-box, a position mirrors an earlier one whose Z-value is known, so most of $Z[i]$ is free. Only growth past the box costs comparisons, and those advance the box – bounding total work linearly.

Algorithm

1. Form $w = pattern + \# + text$; compute its Z-array with the Z-box recurrence.
2. Every index i in the text region with $Z[i] = |pattern|$ is an occurrence.

Dry run

Dry Run. pattern "ab", text "abcab"; $w = "ab\#abcab"$.

- $Z = 2$ at the two positions where "ab" begins in the text.
- Matches at text indices 0 and 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> zFunction(const string& w) {
5     int n = w.size();
6     vector<int> z(n, 0);
7     z[0] = n;
8     int l = 0, r = 0;
9     for (int i = 1; i < n; i++) {
10         if (i < r) z[i] = min(r - i, z[i - l]); // mirror inside the
11             Z-box
12         while (i + z[i] < n && w[z[i]] == w[i + z[i]]) z[i]++;
13         if (i + z[i] > r) { l = i; r = i + z[i]; }
14     }
15     return z;
16 }
17
18 int main() {
19     string pat = "ab", text = "abcab";
20     string w = pat + "#" + text;

```

```

20     vector<int> z = zFunction(w);
21     int m = pat.size();
22     for (int i = m + 1; i < (int)w.size(); i++)
23         if (z[i] == m) cout << i - m - 1 << " ";    // text index
24     cout << "\n";                                     // 0 3
25     return 0;
26 }

```

Complexity Analysis

Time Complexity. $O(n + m)$. **Space Complexity.** $O(n + m)$.

Key Takeaway

The Z-function computes prefix self-matches in linear time via the Z-box, and pattern matching drops out by prepending the pattern with a separator. It is interchangeable with KMP; keep both as linear matchers.

2.19 KMP Algorithm and the LPS Array

Problem Statement. Compute the longest-proper-prefix-which-is-also-suffix (LPS/prefix-function) array of a pattern, and use it to find all occurrences of the pattern in a text in linear time.

Pattern Recognition

KMP avoids re-comparing characters after a mismatch by precomputing the **LPS array**: for each pattern prefix, the length of its longest border. On a mismatch, the pattern shifts so the matched border aligns, never moving the text pointer backward.

Borders let you resume, not restart

The LPS array $\pi[i]$ is the longest proper prefix of $pattern[0..i]$ that is also a suffix. Build it by extending or falling back along previously computed borders. During matching, when a mismatch occurs after matching j characters, instead of restarting you set $j = \pi[j - 1]$ – the next-longest border already matched – and try again. The text pointer only moves forward, giving $O(n + m)$.

The LPS Says Where to Resume After a Mismatch

A border of the matched portion is guaranteed to align with the text, so after a mismatch KMP jumps the pattern to that border rather than re-examining text characters. Precomputing all borders makes each jump $O(1)$ amortised.

Algorithm

1. Build π for the pattern: extend on match, fall back via $\pi[j - 1]$ on mismatch.
2. Scan the text with a pointer j into the pattern; on match advance both, on mismatch set $j = \pi[j - 1]$.
3. When j reaches $|pattern|$, record an occurrence and set $j = \pi[j - 1]$.

Dry run

Dry Run. pattern "aba", text "ababa".

- $\pi = [0, 0, 1]$. Matching finds "aba" at index 0; after a match, fall back to reuse the overlap and match again at index 2.
- Occurrences at 0 and 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> buildLPS(const string& p) {
5     int m = p.size();
6     vector<int> pi(m, 0);
7     for (int i = 1; i < m; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && p[i] != p[j]) j = pi[j - 1];    // fall back on
10                mismatch
11         if (p[i] == p[j]) j++;
12         pi[i] = j;
13     }
14     return pi;
15 }
16
17 vector<int> kmp(const string& text, const string& pat) {
18     vector<int> pi = buildLPS(pat), res;
19     int n = text.size(), m = pat.size(), j = 0;
20     for (int i = 0; i < n; i++) {
21         while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
22         if (text[i] == pat[j]) j++;
23         if (j == m) { res.push_back(i - m + 1); j = pi[j - 1]; }
24     }
25     return res;
26 }
27
28 int main() {
29     for (int p : kmp("ababa", "aba")) cout << p << " ";
30     cout << "\n";    // 0 2
31 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – LPS build and matching are each linear. **Space Complexity.** $O(m)$ for the LPS array.

Key Takeaway

KMP precomputes borders (the LPS array) so a mismatch resumes from the longest overlap instead of restarting, keeping the text pointer moving forward. The LPS array itself answers border/period questions and powers the next two problems.

2.20 Shortest Palindrome

Problem Statement. Given a string s , prepend the minimum number of characters to its front to make the whole string a palindrome. Return the resulting shortest palindrome.

Pattern Recognition

The goal is to find the **longest palindromic prefix** of s ; whatever follows it must be mirrored and prepended. That longest palindromic prefix is found with the **KMP prefix function** applied to $s + \# + reverse(s)$.

KMP finds the longest palindromic prefix

Form $w = s + \# + reverse(s)$ with a separator $\#$ not in s . The prefix function's last value, $\pi[\text{end}]$, is the length of the longest prefix of s that matches a suffix of $reverse(s)$ – which is exactly the longest palindromic prefix of s . The characters of s beyond that prefix, reversed, are what must be prepended to complete the palindrome.

Match s Against Its Reverse

A prefix of s is a palindrome iff it equals the corresponding suffix of $reverse(s)$. Concatenating with a separator and taking the prefix function's final value measures this overlap in linear time, avoiding an $O(n^2)$ palindrome scan.

Algorithm

1. Build $w = s + \# + reverse(s)$; compute its prefix function.
2. Let $k = \pi[\text{last}]$ be the longest palindromic prefix length.
3. Prepend $reverse(s[k :])$ to s and return it.

Dry run

Dry Run. $s = \text{"aacecaaa"}$.

- Longest palindromic prefix is "aacecaa" (length 7); leftover "a".
- Prepend reverse of "a" $\rightarrow$ "aaacecaaa".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> prefixFunction(const string& p) {
5     int m = p.size();
6     vector<int> pi(m, 0);
7     for (int i = 1; i < m; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && p[i] != p[j]) j = pi[j - 1];
10        if (p[i] == p[j]) j++;
11        pi[i] = j;
12    }
13    return pi;
14 }
```

```

15
16 string shortestPalindrome(const string& s) {
17     string rev(s.rbegin(), s.rend());
18     string w = s + "#" + rev;
19     vector<int> pi = prefixFunction(w);
20     int k = pi.back(); // longest
        palindromic prefix
21     return rev.substr(0, s.size() - k) + s; // prepend the
        mirrored rest
22 }
23
24 int main() {
25     cout << shortestPalindrome("aacecaaa") << "\n"; // aaacecaaa
26     cout << shortestPalindrome("abcd") << "\n"; // dcbaabcd
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one prefix-function pass. **Space Complexity.** $O(n)$.

Key Takeaway

Shortest palindrome reduces to the longest palindromic prefix, found by running KMP on $s + \# + \text{reverse}(s)$. The trick of matching a string against its reverse via the prefix function recurs in palindrome and border problems.

2.21 Longest Happy Prefix

Problem Statement. A “happy prefix” is a non-empty prefix of a string that is also a suffix (excluding the whole string). Return the longest happy prefix.

Pattern Recognition

The longest happy prefix is precisely the longest proper border of the string – which is the **last value of the KMP prefix function**. No extra work beyond building the LPS array is needed.

The border is the answer

The prefix function $\pi[i]$ gives the longest proper border of the prefix ending at i . For the whole string, $\pi[n - 1]$ is the length of its longest proper prefix that is also a suffix – the definition of the longest happy prefix. So compute the LPS array and return the prefix of that length.

Longest Happy Prefix = Longest Border = $\pi[n - 1]$

Prefix-that-is-also-suffix is exactly “border,” and the prefix function’s final entry stores the longest border of the entire string. The whole problem is a single LPS computation and a substring.

Algorithm

1. Compute the prefix function π of the string.
2. Return the prefix of length $\pi[n - 1]$.

Dry run

Dry Run. "ababab".

- $\pi = [0, 0, 1, 2, 3, 4]$; $\pi[5] = 4$.
- Longest happy prefix is "abab" (length 4).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string longestPrefix(const string& s) {
5     int n = s.size();
6     vector<int> pi(n, 0);
7     for (int i = 1; i < n; i++) {
8         int j = pi[i - 1];
9         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
10        if (s[i] == s[j]) j++;
11        pi[i] = j;
12    }
13    return s.substr(0, pi[n - 1]);           // longest border
14 }
15
16 int main() {
17     cout << "[" << longestPrefix("ababab") << "]\n";    // [abab]
18     cout << "[" << longestPrefix("leetcode") << "]\n";  // []
19     return 0;
20 }

```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ for the prefix-function array.

Key Takeaway

The longest happy prefix is just the last value of the KMP prefix function – the longest border of the whole string. Recognising “prefix that is also a suffix” as a border collapses the problem to one LPS computation.